MDPI

Article

Hierarchical Scheduler with Adaptive Time-Budget Reallocation for Time-Triggered Edge-Fog-Cloud Architectures

Omar Hekal [1,*], Josepaul Paulachan [2], Daniel Onwuchekwa [3] and Roman Obermaisser [4]

Chair of Embedded Systems, University of Siegen, Hölderlinstraße 3, 57076 Siegen, Germany; josepaul.paulchan@uni-siegen.de (J.P.);daniel.onwuchekwa@uni-siegen.de (D.O.); roman.obermaisser@uni-siegen.de (R.O.)
* Correspondence: omar.hekal@uni-siegen.de

Abstract

The lack of determinism restricts the integration of safety-critical applications into edge–fog–cloud (EFC) architectures. Existing EFC schedulers are typically designed for dynamic, best-effort operation based on unmanaged resource allocation and elastic virtualization. This paradigm introduces unbounded queueing, resource contention, and timing jitter, making standard schedulers unsuitable for hard-deadline workloads. Moreover, most approaches focus on computational placement, while communication is abstracted or treated as a secondary cost term. As a result, bounded-latency routing and deterministic task execution are rarely co-optimized under a unified timing model. This paper addresses these gaps by utilizing a managed Time-Triggered Edge–Fog–Cloud (TTEFC) architecture that supports safety-critical workloads, orchestrates IEEE Time-Sensitive Networking (TSN) for local intra-domain communication, and uses IETF Deterministic Networking (DetNet) for routed inter-domain paths. On this infrastructure, a Hierarchical Genetic Algorithm (HGA) is proposed to jointly schedule partition-to-cluster mapping, partition execution order, inter-partition route selection, and negotiated per-partition time budgets that act as temporal boundaries for parallel partition-level optimizers. An adaptive slack reallocation operator redistributes unused temporal slack from over-satisfied partitions to budget-violating partitions, improving feasibility convergence. Experiments on synthetic DAG workloads with 100–500 tasks compare the proposed HGA against HEFT and round-robin baselines. Under the evaluated platform and deterministic-network assumptions, HGA reduces global makespan by up to 46.5% and 18.9% relative to HEFT and round-robin, respectively, while satisfying all baseline deadline and partition-budget constraints. Ablation results show that slack reallocation improves partition-budget feasibility, reaches feasible budget assignments earlier, and produces tighter budget–makespan alignment than feedback-free and static-budget variants. An automotive-characteristic DAG case study further demonstrates feasibility on an application-oriented workload.

Keywords: Algorithms, Cloud Architectures, Cloud Scheduling, Co-optimization, Distributed Systems, Genetic Algorithms, Hierarchical Scheduling, Multi-tier Architecture, Optimization, Partition Negotiation, Scheduling.

Received:
Revised:
Accepted:
Published:

1. Introduction

The Edge–Fog–Cloud (EFC) continuum is increasingly adopted as an architectural model for distributed, data-intensive applications [1]. By distributing computation, storage, and networking capabilities from centralized cloud resources toward the network edge, EFC

enables scalable processing of the massive data streams generated by the Internet of Things (IoT). In parallel, safety-critical domains such as industrial automation, automotive systems, and avionics, which are characterized by stringent timing constraints, are also undergoing data-centric transformations [2]. Traditionally deployed on specialized platforms, these systems are facing new demands for scalability and connectivity, as well as lifecycle challenges associated with rapid hardware obsolescence. Migrating safety-critical logic onto distributed EFC infrastructures promises substantial benefits. However, it exposes a fundamental conflict between best-effort EFC design principles and the determinism required by safety-critical systems. This conflict is referred to as the determinism gap: safety-critical workloads require strict end-to-end timing guarantees that state-of-the-art EFC architectures were not originally designed to provide [3].

The primary value proposition of many Edge–Fog–Cloud (EFC) research efforts is their dynamism, including flexible resource allocation [4] and adaptation to fluctuating workloads [5]. While this flexible model is attractive for best-effort distributed applications such as streaming analytics [6], it is problematic for safety-critical applications governed by hard end-to-end deadlines. In high-integrity real-time domains, including industrial automation, automotive systems, and avionics, temporal correctness is as important as functional correctness; a computed control command delivered after its deadline may be functionally useless and potentially unsafe [7,8]. Because unmanaged public cloud environments include resource contention and virtualization effects that are not bounded at the application level, executing safety-critical logic on standard EFC infrastructures shifts critical timing risks to runtime layers outside the scheduler's control, thereby complicating the repeatable timing behavior required for high-integrity safety-critical applications [9,10].

This determinism gap is further amplified at the communication level. Standard EFC deployments commonly rely on best-effort communication protocols and routed application-layer messaging, which do not by themselves provide strict bounds on message latency [11]. Without bounded communication delays and synchronized time references, timing guarantees across a distributed multi-tier continuum cannot be established within the scheduling model.

This work addresses the determinism gap by adopting the time-triggered design principles [8,12] within the Time-Triggered Edge–Fog–Cloud (TTEFC) architectural model introduced in our prior work [13]. The TTEFC model targets managed, deterministic EFC systems in which temporal isolation is provisioned rather than assumed. Under this model, compute resources provide bounded task execution behavior, while the network fabric exposes a Deterministic Communication Interface (DCI) for bounded-latency communication [14]. The DCI abstracts deterministic communication services that can be implemented using IEEE Time-Sensitive Networking (TSN) in local Layer 2 domains and IETF Deterministic Networking (DetNet) in routed Layer 3 segments [15,16], together with managed time-synchronization support such as emerging 5G Time Synchronization as a Service (TSaaS) [17]. DCI compliance is therefore treated as an infrastructure-level responsibility of the provider or network operator.

Within this scoped timing model, the remaining challenge is at the scheduler level: the system must synthesize an offline end-to-end allocation and execution plan that jointly accounts for task execution, inter-partition communication, and deadline feasibility. This problem is NP-hard [18]. Existing EFC and workflow schedulers, including classical baselines such as HEFT [19] and GA-based EFC schedulers [3], do not explicitly combine deterministic route selection with feedback-driven time-budget adjustment. Accordingly, two scheduler-level gaps are identified.

First, computation placement and communication routing are commonly treated as separate, often sequential, optimization problems [3]. In typical EFC baselines, optimization

is primarily driven by compute-centric objectives such as makespan, while communication is represented by simplified delay terms rather than explicit deterministic route selection. This separation can produce placements that appear computationally efficient but still lead to infeasible routed communication delays or to contention-sensitive communication patterns that are not visible to the scheduler.

Second, adaptive time-budget adjustment across partitions is not explicitly modeled. A partition may exceed its assigned budget while another retains unused slack. Yet conventional hierarchical or decoupled scheduling flows lack an explicit feedback mechanism to redistribute this slack during optimization. As a result, feasibility recovery is often left to unguided perturbations, which can slow convergence or fail to find feasible end-to-end solutions even when sufficient global timing margin exists.

This paper addresses these two gaps by focusing on offline temporal feasibility synthesis over heterogeneous compute resources and bounded TSN/DetNet communication paths. In particular, the proposed HGA extends the earlier TTEFC optimization model [13] by coupling partition allocation, partition execution order, deterministic inter-partition route selection, and per-partition time-budget adjustment into a single feasibility-aware search. At the system level, the chromosome jointly encodes partition-to-cluster allocation, partition execution order, route-selection hints, and time budgets. At the partition level, local GAs synthesize task schedules under the assigned time-budget constraints and return realized makespans and budget violations. The adaptive slack reallocation operator then uses these feedback values to shift unused temporal budget toward partitions that require additional execution margin. The evaluation includes an ablation study of adaptive time-budget reallocation, deadline-sensitivity analysis, and an automotive-characteristic case study. Table 1 summarizes the main differences between the prior work [13] and the present method.

Table 1. Comparison between the prior TTEFC framework [13] and the present method.

Aspect	Prior TTEFC framework [13]	Present method
Scheduling focus	Spatial task-to-tier partitioning followed by local scheduling within each tier.	Feasibility-aware hierarchical scheduling with partition allocation, execution ordering, time-budget assignment, and route selection.
Partition coordination	Local schedules are optimized independently after partitioning.	Partition schedules are coordinated through system-level time budgets and feedback from realized partition makespans.
Communication treatment	Communication is represented through static delay costs.	Deterministic communication paths are selected and included during end-to-end schedule reconstruction.
Temporal feasibility	No explicit partition-level time-budget adaptation is used.	Partition time budgets are adapted by reallocating available slack to budget-violating partitions.
Main objective	Reduce spatial partitioning and local scheduling cost.	Synthesize schedules that satisfy both partition-level budgets and the global deadline.

The contributions of this work are fourfold:

- **A hierarchical GA for temporal and routing co-design.** A two-level HGA is proposed in which the system-level GA jointly optimizes partition-to-cluster allocation, partition execution order, deterministic inter-partition route-selection hints, and per-partition time budgets. These decisions constrain parallel partition-level GAs that synthesize

local task schedules under assigned time-budget constraints, enabling coordinated end-to-end optimization across system and partition layers.

- **Adaptive time-budget reallocation for hierarchical feasibility.** A feedback-guided slack reallocation operator is introduced to adjust partition time budgets during the system-level search. The operator uses realized partition makespans, budget violations, and available slack to transfer temporal margin from over-satisfied partitions to partitions under timing pressure, thereby improving partition-budget feasibility and convergence stability.
- **Integrated deterministic routing within the scheduling search.** The proposed framework embeds route-selection decisions into the system-level chromosome, linking partition placement and scheduling decisions with bounded TSN/DetNet communication paths. This enables communication infeasibility to be considered during schedule reconstruction rather than treated as an external post-processing step.
- **Evaluation through benchmarks, ablation, and an automotive-characteristic case study.** The proposed scheduler is evaluated on scalable synthetic DAG workloads, deadline-tightening scenarios, and an ablation study comparing feedback-guided reallocation against feedback-free and static-budget variants. In addition, an automotive-characteristic case study is constructed from public benchmark characteristics to assess the method on an application-oriented workload structure.

The remainder of this paper is organized as follows. Section 2 reviews the related works. Section 3 introduces the time-triggered edge–fog–cloud system model and formulates the scheduling problem. Section 4 presents the proposed hierarchical scheduling framework. Section 5 evaluates the proposed approach through benchmark and sensitivity analyses. Section 6 discusses the main observations derived from the results. Section 7 concludes the paper.

2. Related Works

Modern IoT applications are increasingly deployed across an edge–fog–cloud (EFC) continuum to meet demands for low latency and high throughput while exploiting heterogeneous resources near data sources. This shift motivates architectural models that clarify the roles of edge, fog, and cloud nodes, and it raises system-level challenges in coordinating computation and communication across tiers. As this work addresses scheduling and communication-aware task placement in EFC settings, the following discussion will focus on architectural concepts and abstractions that directly affect timing, resource availability, and orchestration constraints.

2.1. Edge-Fog-Cloud Architectures

EFC architectures extend cloud-centric deployments by distributing computation across edge and fog layers to improve responsiveness and reduce traffic. In [20], an EFC-based framework for streaming analytics was introduced to support IoT scenarios where continuous data streams and fast reaction times make purely centralized processing impractical. Architectural resilience and operational robustness have also been emphasized in EFC ecosystems. A resilience-oriented taxonomy was provided in [21], where approaches were grouped into capacity-, trustworthiness-, and efficiency-driven mechanisms, highlighting that orchestration policies should reflect node capabilities and failure characteristics rather than relying solely on offloading models.

The increasing adoption of IoT–fog–cloud ecosystems was further surveyed in [22], where fog was described as an intermediate layer that complements the cloud by enabling localized analytics and security functions while supporting quality of service objectives. Standardization efforts have aimed to formalize these layered deployments, including

OpenFog/IEEE 1934 guidelines [23] and MEC reference models [24]. Overall, these works motivate multi-tier abstractions in which heterogeneous resources, locality, and tier-specific constraints jointly shape feasible orchestration and scheduling decisions.

2.2. Communication in Edge-Fog-Cloud Architectures

Although deterministic communication is central to this work, much of the IoT-edge-cloud literature focuses on protocol heterogeneity and security in best-effort environments. Application-layer surveys emphasize that different tiers often adopt different protocols depending on latency and energy constraints [11], while security frameworks focus on scalable trust and secure data exchange under dynamic connectivity [25,26]. However, such approaches typically optimize flexibility and integrity rather than providing strict end-to-end timing guarantees, which motivates deterministic communication fabrics (e.g., TSN/DetNet) where transmission is treated as a schedulable resource.

Deterministic networking is often paired with centralized orchestration (e.g., SDN controllers) because routing decisions and TSN/DetNet schedule configuration must be coordinated to satisfy timing constraints. Zhang et al.[27] proposed an SDN-assisted deterministic architecture that couples TSN configuration with centralized control, highlighting that routing and scheduling must be co-designed to meet timing requirements under mixed traffic. This observation is consistent with the present work, where communication choices are integrated into the scheduling search to avoid network contention that may violate end-to-end deadlines.

Nie et al.[28] investigated joint communication and computation coordination for TSN-based edge environments using dependency models to ensure deterministic completion across multiple hops. While these studies primarily rely on heuristic planning of transmission and offloading decisions, the present work emphasizes hierarchical time-budget negotiation and introduces a slack reallocation mechanism to rebalance temporal budgets when local violations occur.

2.3. Cloud Scheduling

2.3.1. Heuristic-based Scheduling

Heuristic and meta-heuristic scheduling have been widely used in cloud environments to solve NP-hard workflow and resource allocation problems, typically minimizing makespan and cost while maintaining balanced resource utilization. Hybrid meta-heuristics have been proposed to improve solution quality and convergence behavior by combining complementary search strategies. For example, Ant-Lion Optimization combined with Particle Swarm Optimization was applied to multi-objective workflow scheduling [29], and hybrid formulations integrating Genetic Algorithms, Differential Evolution, and Simulated Annealing were evaluated for large task sets to reduce completion time [30]. Genetic-Algorithm-based schedulers have also been explored for deadline-aware task execution in cloud-based real-time AI workloads [31]. More recent hybrid designs further combine multiple operators to mitigate premature convergence and local optima in task scheduling [32].

Survey studies confirm the prevalence of meta-heuristics in cloud scheduling and show that makespan, cost, and load balancing remain the dominant optimization criteria [33]. Despite their effectiveness, heuristic and meta-heuristic methods may suffer from parameter sensitivity, scalability limitations in high-dimensional decision spaces, and the risk of local-optimal solutions, particularly under multiple conflicting objectives. These limitations motivate scheduling frameworks that explicitly couple computation and communication decisions under timing constraints, particularly in deterministic edge–fog–cloud environments.

Learning- and data-driven schedulers have also been explored to react to workload dynamics or to approximate expensive optimization decisions, for example using deep reinforcement learning and learned surrogate models [34–37]. Since the present work targets deterministic end-to-end timing and does not employ learning-based policies, the subsequent discussion emphasizes reservation-based and hierarchical models that provide explicit timing guarantees.

2.3.2. Reservation-Based and Hierarchical Scheduling

In EFC deployments, predictability is often pursued through reservation-based and hierarchical scheduling to enforce temporal isolation and limit interference among co-located applications. Recent work extends container orchestration with real-time reservations and introduces online admission tests and monitoring loops to manage workload variability at the edge.

KubeDeadline [38] extends Kubernetes with SCHED-DEADLINE-based reservations and node-level admission control, enabling isolated execution of latency-sensitive containerized workloads under interference. Online admission control has also been studied for multicore platforms with non-strictly periodic arrivals, where arrival-curve models and reachability-based analyses are used to decide whether new jobs can be admitted without violating existing deadlines [39]. Hierarchical orchestration approaches combine offline placement/dimensioning with online controllers that monitor deadline misses and response times and adjust CPU budgets or trigger migrations when local adaptation is insufficient [40]. For workloads with highly variable execution times, job-acceptance and dismissal policies have been proposed to bound queue growth and control miss rates under probabilistic execution-time estimates [41]. Complementary monitoring mechanisms dynamically reconfigure SCHED_DEADLINE budgets based on observed runtime behavior to balance isolation with utilization [42].

Unlike these largely reactive or admission-driven frameworks, the present work targets deterministic timing through offline schedule synthesis under explicit computation and communication constraints. In particular, partition placement is coupled with deterministic TSN/DetNet routing through communication-aware chromosomes, and budget negotiation is formalized via an Adaptive Slack Reallocation Operator that transfers temporal slack between partitions to restore feasibility and improve convergence across the multi-tier continuum.

Runtime adaptation and fault tolerance in edge–fog–cloud systems are widely studied, typically through proactive and reactive mitigation actions such as migration, replication, and checkpointing, and through monitoring-driven resource management [43,44]. Adaptive resource allocation schemes further incorporate workload- and failure-aware selection policies to improve robustness under dynamic conditions [45,46]. These directions primarily target service continuity and availability and are outside the scope of this paper. Instead, the focus is placed on temporal determinism, where adaptation is realized at the scheduling level via time-budget negotiation: temporal slack is redistributed between partitions to maintain feasibility under explicit computation and communication constraints.

Within the context of hierarchical scheduling, it is necessary to distinguish the proposed scheduling framework from the preliminary optimization model developed for the TTEFC architecture in our prior work [13]. While [13] established the physical multi-tier TTEFC architectural model, its scheduling approach was restricted to a spatial partitioning paradigm. The nested algorithm in [13] used a global optimization loop to solve the spatial task-to-tier mapping problem using static graph-partitioning cuts, while local loops minimized makespans within independent tiers.

The preliminary scheduling model did not consider partition-level time budgeting, temporal negotiation, or explicit network routing. Because the tiers operated without local timing boundaries or coordinated temporal feedback, a bottleneck in a single local partition would stall the entire global search, even when adjacent tiers possessed ample timing slack. Furthermore, communication costs were represented as flat, static delay terms, omitting explicit route selection and path pinning on the underlying deterministic TSN and DetNet substrates.

To address these limitations, this work shifts from spatial graph partitioning to a joint temporal-routing co-design paradigm. The proposed HGA introduces a system-level budget vector (TB) to negotiate explicit temporal boundaries between partitions, governed by an offline adaptive slack reallocation operator that dynamically rebalances temporal budgets based on partition-level feedback. Additionally, we integrate deterministic path selection directly into the system-level search space using explicit path-index genes (ρ), co-optimizing task placement with Layer 2 TSN and Layer 3 DetNet routes to eliminate runtime link contention. The next section introduces the system model and problem formulation that capture these coupled constraints.

3. System Model

This section defines the time-triggered Edge–Fog–Cloud (TTEFC) system model considered in this work and the formal abstractions used by the proposed scheduler. The model represents a managed Edge–Fog–Cloud setting in which task execution times, communication delays, and clock offsets are bounded. This defines the timing scope of the scheduling problem studied in this paper and distinguishes it from unmanaged best-effort IoT/cloud environments. We first describe the architecture and clarify the scope of the deterministic timing model. We then define the platform, network, and application models used to formulate the end-to-end scheduling problem. The scheduling algorithms are presented in Section 4.

3.1. TTEFC Architecture and Determinism Assumptions

Our work considers a distributed application deployed across a hierarchical Edge–Fog–Cloud (EFC) continuum. This architecture is driven by the growth of IoT data and the need to meet stringent timing requirements. A fog orchestrator coordinates the execution of application partitions across tiers and manages the timing of inter-partition communication. Cross-domain communication (including Fog–Cloud and Cloud–Cloud connectivity) is assumed to use a deterministic backbone with bounded latency, while the fog and edge layers operate on Time-Sensitive Networking (TSN). TSN extends standard Ethernet with time synchronization, scheduled traffic, and resource reservation, enabling bounded latency under a shared time base. These assumptions enable deterministic system behavior, allowing the orchestrator to make scheduling and routing decisions with predictable execution and communication latencies, an essential requirement for safety-critical applications.

3.2. Global Synchronization and TSaaS

To support time-triggered execution across the distributed continuum, a system-wide global time base is assumed. The 3GPP *Time Synchronization as a Service (TSaaS)* framework (Release 18) [17] facilitates this by leveraging 5G/6G infrastructure to distribute a Grandmaster clock signal with sub-microsecond accuracy to participating nodes. Any residual clock drift and distribution uncertainty are assumed to remain within a bounded error margin ensured by the synchronization service and the underlying network configuration, so that the time base can be used consistently for deterministic communication and time-

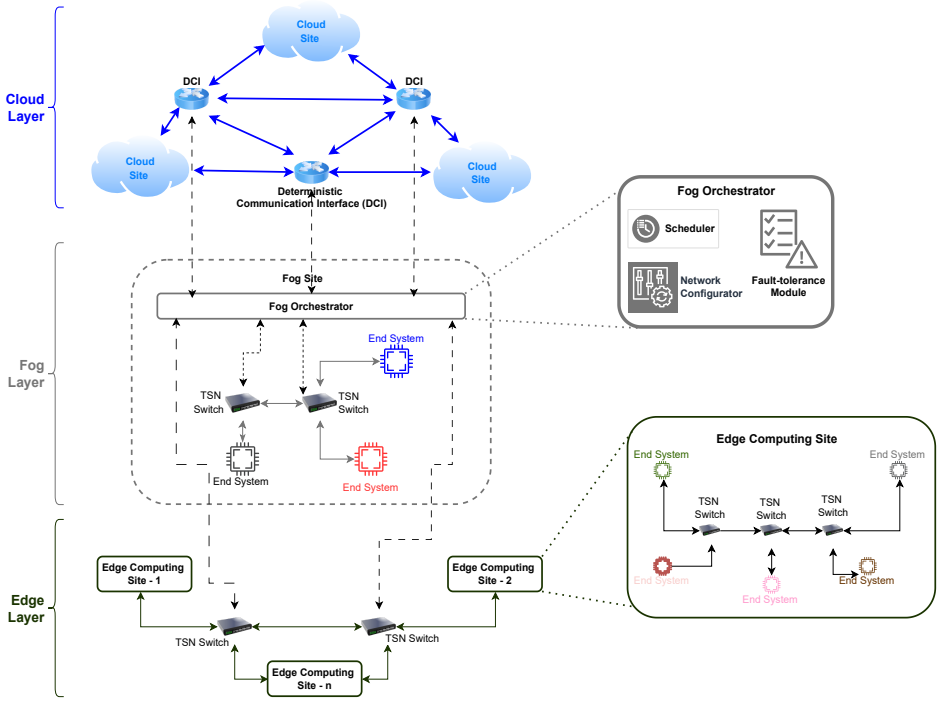

Figure 1. Reference TTEFC deployment architecture model used by the hierarchical scheduler.

triggered execution. This time base is assumed to be available to Edge, Fog, and Cloud end systems in the considered managed deployment

3.3. Edge Layer

The edge layer represents the physical boundary of the system and hosts the most latency-sensitive functions, such as sensing, preprocessing, and closed-loop actuation. Edge end systems are typically resource-constrained and therefore execute lightweight processing close to data sources, while compute-intensive functions may be offloaded to higher tiers when permitted by end-to-end timing constraints. The edge domain follows the TSN assumptions stated in Section 3.1.

3.4. Fog Layer

The fog layer is an intermediate tier between the resource-constrained edge and the high-capacity cloud. It hosts the fog orchestrator, which acts as the application's coordinator by scheduling workloads, negotiating with different computational clusters, and offloading tasks to cloud or edge resources based on timing constraints. Positioned closer to edge devices than the cloud but with greater computational capacity than the edge, the fog layer is well-suited to manage time-critical workloads: it can rapidly react to task arrivals, distribute processing across local nodes, and leverage cloud resources when necessary to meet stringent deadlines. The fog domain follows the TSN assumptions stated in Section 3.1

3.4.1. Scheduler Module

The scheduler module implements a hierarchical scheduling mechanism that synthesizes both system-level and partition-level schedules. At the system level, it coordinates (i) the allocation of application partitions to Edge, Fog, or Cloud resources, (ii) the execution order of these partitions, (iii) the timing of inter-partition communication, and (iv) the assignment of per-partition time budgets that collectively satisfy the end-to-end deadline. At the partition level, each partition is scheduled independently within its assigned time

budget by determining the internal task execution order, mapping tasks to processors, and enforcing intra-partition communication constraints to minimize the local makespan while preserving precedence and resource feasibility.

To improve feasibility under heterogeneous execution and communication delays, the scheduler incorporates an adaptive time-budget negotiation mechanism that redistributes slack from partitions that complete early to those that exceed their allocated budgets. The resulting local makespans are reported to the system level; when a partition exceeds its budget, feasibility is encouraged through targeted penalties and repair actions that guide the search toward schedules that satisfy the global timing constraints.

This hierarchical coordination mitigates globally infeasible solutions that may arise when task placement, timing, and communication routing are optimized independently across tiers. By negotiating time budgets based on feedback from partition schedules, end-to-end determinism and deadline compliance are maintained under varying workload characteristics.

3.4.2. Network Configurator

The network configurator provisions deterministic communication resources for application messages at both the inter- and intra-partition levels. For *inter-partition* messages, it maps each flow onto the deterministic communication fabric and reserves the required paths and transmission resources to maintain bounded latency. It exposes the set of feasible deterministic routes between communicating endpoints and associates each route with a bounded latency that is used during schedulability analysis. If no feasible deterministic connection exists for a required sender–receiver pair, the corresponding communication is treated as infeasible and handled as a violation in the end-to-end feasibility assessment.

For *intra-partition* messages, the configurator provides the bounded communication parameters required by the partition-level scheduler. When communicating tasks are placed on the same end system, the transfer is abstracted as a local delivery with negligible network delay. Otherwise, the transfer is constrained to the local deterministic domain (e.g., TSN) and is modeled with a bounded latency consistent with the provisioned local communication resources.

3.4.3. Fault-Tolerance Module

The architecture anticipates deterministic adaptation by integrating concepts from multi-schedule graphs (MSGs) and metascheduling. In time-triggered systems, metascheduling computes a schedule for each possible context event and organizes the schedules into a multi-schedule graph. Upon the occurrence of an event (e.g., a resource failure or detected slack), the system switches from the active schedule to the appropriate successor schedule in the MSG. Recent work in [2] shows that a multi-objective genetic algorithm-based metascheduler can generate MSGs while minimizing memory consumption by exploiting structural similarity between schedules and storing only their incremental differences. By relying on pre-computed schedule variants and deterministic schedule switching, fault handling and recovery can be performed without introducing runtime uncertainty that could compromise end-to-end timing guarantees. This module is included as an architectural extension point; the evaluation in this paper focuses on baseline schedule synthesis.

3.5. Cloud Layer

Unlike standard best-effort cloud models (e.g., AWS, GCP), the Cloud Layer in the proposed TTEFC architecture is defined as a *Managed Real-Time Cloud Slice*. This represents a logical partitioning of data center resources, with temporal isolation strictly enforced by real-time hypervisors and partitioning kernels. This substrate ensures a bounded Worst-Case Execution Time (WCET) for offloaded tasks. While our model uses representative

resource constraints inspired by commercial platforms, the scheduler's ability to provide timing guarantees relies on this deterministic compute substrate.

3.6. Deterministic Communication Interface (DCI)

The deterministic backbone used for inter-site connectivity is modeled as a *Deterministic Communication Interface (DCI)*. The DCI is assumed to provide bounded-latency communication for cross-domain transfers, including Fog–Cloud and Cloud–Cloud connectivity. This interface is implemented using IETF DetNet flows, which enable explicit route pinning and resource reservation to guarantee bounded latency [16]. Our model accounts for the significant propagation delay of these backbone paths by assigning higher communication costs to DCI transfers compared to local Edge/Fog links operating within TSN domains.

3.7. End Systems in the TTEFC Architecture

In the proposed architecture, the term End System (ES) is used in the strict sense defined by the avionics (TTEthernet) and industrial automation (TSN) domains. An End System is not merely a passive communication interface; it is a computational node equipped with a deterministic network interface. An End System ES_i is a tuple (C_i, N_i), where C_i represents the Computational Unit, comprising the CPU (Host), memory, and operating system capable of executing application tasks, and N_i represents the Network Unit, specifically a TSN-compliant Network Interface Card (NIC) or a specialized network controller.

The ES facilitates the interaction between the application layer and the deterministic network fabric through three primary functional roles:

- **Computation:** It executes the tasks assigned to it by the scheduler.
- **Traffic Injection (Talker):** It injects data messages into the network at precise, pre-scheduled time instants governed by the global schedule. The Network Unit N_i enforces traffic shaping (e.g., using IEEE 802.1Qbv time-aware shapers) to ensure that traffic enters the switch fabric without inducing congestion [47].
- **Traffic Reception (Listener):** It receives messages from the network, buffering them for consumption by the local tasks.

3.7.1. Heterogeneity of End Systems in the TTEFC Architecture

To capture the heterogeneous nature of the continuum, End Systems (ES) are differentiated across the Edge, Fog, and Cloud layers.

Edge End Systems

Edge End Systems represent the physical boundary of the system "far edge" and include smart sensors, Programmable Logic Controllers (PLCs), robotic joint controllers, and automotive Electronic Control Units (ECUs). These nodes are characterized by constrained computational resources. Typically based on microcontrollers (e.g., ARM Cortex-M/R) or low-power Systems-on-Chip (SoCs), they provide limited RAM and processing capability. Consequently, while they offer negligible communication latency for local I/O interactions, their processing-time values for complex tasks remain comparatively high. Their primary network role is to serve as sources of raw data streams and sinks for actuation commands.

Fog End Systems

Fog End Systems act as intermediate aggregation and processing points at the "near edge." These nodes, exemplified by Industrial PCs (IPCs), edge gateways, and roadside units (RSUs), offer moderate computational capabilities that are significantly higher than those of edge nodes. For the scheduler, fog nodes represent a critical trade-off point: they provide execution speeds capable of handling tasks that are heavy for edge nodes while

maintaining lower communication latencies than the cloud. Architecturally, they bridge local TSN domains at the edge with the routed deterministic backbone leading to the cloud.

Cloud End Systems

Cloud End Systems constitute the high-performance tier located in data centers, comprising rack-mounted servers and bare-metal instances. These nodes provide massive memory bandwidth and server-grade processing power (e.g., Xeon and EPYC CPUs and GP-GPUs), resulting in the lowest processing-time values for computationally intensive workloads such as global path planning and model training. However, accessing these resources incurs a significant latency penalty due to physical distance and routing hops.

3.8. Platform and Network Model

The execution platform is modeled as a set of heterogeneous End Systems distributed across the Edge, Fog, and Cloud layers, interconnected via deterministic communication resources. Let $\mathcal{N} = \{ES_1, ES_2, \ldots, ES_{|\mathcal{N}|}\}$ denote the set of End Systems distributed across the Edge, Fog, and Cloud tiers. The network topology is extracted from the platform description and modeled as a graph $\mathcal{G}_{net} = (\mathcal{V}, \mathcal{L})$, where $\mathcal{V}$ contains End Systems and intermediate forwarding elements, and $\mathcal{L}$ contains the communication links between them. For any sender–receiver pair $(s, r) \in \mathcal{N} \times \mathcal{N}$, a finite set of feasible candidate paths is derived from $\mathcal{G}_{net}$ and denoted by $\mathcal{P}_{s,r} = \{p_1, p_2, \ldots\}$; the path construction procedure is described in Section 4. The Edge/Fog domains follow the TSN assumptions of Section 3.1, while cross-domain transfers rely on the deterministic backbone modeled as the DCI (Section 3.6).

For any sender–receiver pair $(s, r) \in \mathcal{N} \times \mathcal{N}$, a finite set of feasible deterministic paths is assumed to exist, denoted by $\mathcal{P}_{s,r} = \{p_1, p_2, \ldots\}$. For a message (m) routed over a selected path $p \in \mathcal{P}_{s,r}$, the end-to-end communication latency is modeled by a bounded cost function $L(m, p)$ that captures the propagation delay of the selected route and the message-dependent transmission effects. Consistent with Section 3.6, paths traversing the DCI are modeled with higher latency than local Edge/Fog paths.

3.9. Application Model

The application workload is modeled as a dependency graph, where data flow determines execution eligibility. The application $\mathcal{A}$ is represented as a Directed Acyclic Graph (DAG) $\mathcal{A} = (\mathcal{T}, \mathcal{E})$, where $\mathcal{T}$ is the task set and $\mathcal{E}$ is the set of directed dependencies.

3.9.1. Task Set ($\mathcal{T}$)

Let $\mathcal{T} = \{t_1, t_2, \ldots, t_n\}$ be a finite set of tasks. Each task t_i is a non-preemptable unit of execution. Task execution times depend on the selected End System; let $PT(t_i, ES_k)$ denote the processing time (e.g., WCET) of task t_i when executed on $ES_k \in \mathcal{N}$.

3.9.2. Dependency Set ($\mathcal{E}$)

Let $\mathcal{E} \subseteq \mathcal{T} \times \mathcal{T}$ denote the set of directed edges representing precedence and data dependencies. An edge $(t_i, t_j) \in \mathcal{E}$ indicates that t_j cannot start until t_i completes and the produced data is delivered to the End System executing t_j.

3.9.3. Partition Model ($\mathcal{P}$)

To support modularity and hierarchical scheduling, the application is decomposed into a set of partitions, each grouping a subset of tasks scheduled together at the partition level. Let $\mathcal{P} = \{p_1, p_2, \ldots, p_z\}$ denote the set of partitions, and let $Part(t) \in \mathcal{P}$ return the partition to which a task $t \in \mathcal{T}$ belongs.

For a given partition $p \in \mathcal{P}$, the corresponding task subset is:

$$\mathcal{T}_p = \{\, t \in \mathcal{T} \mid Part(t) = p \,\}. \tag{1}$$

The dependencies internal to partition p are:

$$\mathcal{E}_p = \{\, (u,v) \in \mathcal{E} \mid u \in \mathcal{T}_p \wedge v \in \mathcal{T}_p \,\}. \tag{2}$$

Accordingly, the intra-partition workload is represented by the subgraph $\mathcal{G}_p = (\mathcal{T}_p, \mathcal{E}_p)$.

3.9.4. Message Sets ($\mathcal{M}$)

Communication is modeled explicitly as messages associated with dependency edges. For each edge $(u,v) \in \mathcal{E}$, a message $m_{u,v}$ represents the data transfer from the producer task u to the consumer task v. The set of all application messages is:

$$\mathcal{M} = \{\, m_{u,v} \mid (u,v) \in \mathcal{E} \,\}. \tag{3}$$

Inter- and intra-partition messages are distinguished based on whether the communicating tasks belong to different partitions:

$$\mathcal{M}_{inter} = \{\, m_{u,v} \in \mathcal{M} \mid Part(u) \neq Part(v) \,\}, \tag{4}$$

$$\mathcal{M}_{intra} = \{\, m_{u,v} \in \mathcal{M} \mid Part(u) = Part(v) \,\}. \tag{5}$$

Messages in $\mathcal{M}_{inter}$ traverse the deterministic communication fabric across tiers, while messages in $\mathcal{M}_{intra}$ are handled within the scope of a single partition.

3.10. Problem Definition

Given an application DAG $\mathcal{A} = (\mathcal{T}, \mathcal{E})$, a partition set $\mathcal{P}$, a heterogeneous set of End Systems $\mathcal{N}$ distributed across the Edge, Fog, and Cloud tiers, and deterministic path catalogs $\{\mathcal{P}_{s,r}\}$, the goal is to synthesize an end-to-end schedule that jointly accounts for (i) system-level partition decisions across tiers and (ii) partition-level task schedules under bounded execution and communication latencies. A global end-to-end deadline D is specified for the application.

3.10.1. Decision Variables

The following coupled decisions determine the end-to-end schedule:

- **System-level partition allocation:** a mapping $\Phi : \mathcal{P} \to \mathcal{C}$ that assigns each partition p to a selected execution location (cluster/tier endpoint) $\Phi(p)$ within the Edge–Fog–Cloud continuum. This decision fixes the partition's execution domain at the system level.
- **Partition-level task mapping:** for each partition p, a local mapping $\phi_p : \mathcal{T}_p \to \mathcal{N}_{\Phi(p)}$ that assigns the tasks of p to End Systems available within the domain selected by $\Phi(p)$. This decision is produced by the partition-level scheduler, subject to the budget and feasibility constraints imposed by the system-level.
- **System-level execution order:** an ordered sequence π over $\mathcal{P}$ that defines the system-level order in which partitions are integrated into the global schedule.
- **Deterministic routing for inter-partition messages:** for each inter-partition message $m_{u,v} \in \mathcal{M}_{inter}$, a deterministic path $\rho(m_{u,v}) \in \mathcal{P}_{s,r}$ is selected, where $s = \phi_{Part(u)}(u)$ and $r = \phi_{Part(v)}(v)$ denote the End Systems executing the sender task u and the receiver task v, respectively.

- **Per-partition time budgets:** a budget vector $TB = \{TB(p) \mid p \in \mathcal{P}\}$ that assigns an execution-time budget constraint to each partition. The budgets are negotiated so they support the feasibility of the global deadline.

3.10.2. Constraints

Feasibility of an end-to-end schedule requires:

- **Precedence and data availability:** for every dependency edge $(u, v) \in \mathcal{E}$, task v must not start before task u completes and the associated message $m_{u,v}$ is delivered.
- **Resource feasibility:** tasks mapped to the same End System must not overlap in time on the same processing resource, and each task executes with processing time $PT(t_i, ES_k)$ on its assigned End System.
- **Bounded communication:** each inter-partition message $m_{u,v} \in \mathcal{M}_{inter}$ must be assigned a feasible deterministic path $\rho(m_{u,v})$ with bounded latency $L(m_{u,v}, \rho(m_{u,v}))$; if no feasible path exists, the solution is infeasible.
- **Partition time-budget compliance:** for each partition $p \in \mathcal{P}$, the makespan of its partition-level schedule, denoted by $MS(p)$, must satisfy $MS(p) \leq TB(p)$.
- **End-to-end deadline:** the global completion time (global makespan) MS_{global} must satisfy $MS_{global} \leq D$.

3.10.3. Optimization Objectives

The primary objective is to obtain feasible schedules that meet the end-to-end deadline under bounded execution and communication latencies. Among feasible solutions, the optimization targets reduced end-to-end completion time and improved feasibility margins by (i) minimizing the global makespan MS_{global} and (ii) minimizing cumulative partition budget violations when infeasibilities occur during the search. This formulation motivates a hierarchical solution approach in which system-level decisions (partition placement across tiers, execution order, routing, and time budgets) constrain parallel partition-level schedulers that map and schedule tasks within each selected domain.

4. Scheduling Framework

This section presents the proposed two-level scheduler for the problem defined in Section 3.10.

4.1. Framework Overview

The proposed scheduler is organized into a *system-level scheduler* and multiple parallel *partition-level schedulers*. The system-level scheduler determines (i) partition allocation across execution domains, (ii) the system-level execution order of partitions, (iii) deterministic route selections for inter-partition messages, and (iv) per-partition time budgets that define offline temporal budget constraints. Given $(\Phi(p), TB(p))$, each partition-level scheduler constructs a local schedule *within the assigned time budget* by selecting a precedence-feasible task order and mapping tasks to End Systems within the allocated domain while enforcing intra-partition communication constraints. The resulting partition schedules are integrated through an end-to-end reconstruction step that inserts inter-partition transmissions using the selected deterministic paths with bounded latency; the reconstructed schedule is then evaluated to obtain the global makespan and feasibility violations for each system-level solution.

4.2. System-Level Scheduler

The system-level scheduler evolves a population of candidate end-to-end solutions, where each individual jointly encodes (i) a partition allocation across execution domains,

Algorithm 1 System-level scheduler

Input: Per-partition data $(PT^p, \mathcal{M}^p_{\text{intra}}, JD^p)$; inter-partition messages $\mathcal{M}_{\text{inter}}$; path catalogs $\{\mathcal{P}_{s,r}\}$; global deadline D; GA parameters $(N_{\text{pop}}, N_{\text{gen}}, p_{\text{cx}}, p_{\text{mut}})$
Output: Best evaluated individual $ind^\star$

Genome: $ind = [\,\pi \mid \Phi \mid \rho \mid TB\,]$
$Pop \leftarrow \textsc{InitPopulation}(N_{\text{pop}}, D)$
for all $ind \in Pop$ **do**
  ind.fit $\leftarrow \textsc{Evaluate}(ind)$ ▷ Alg. 2
end for
$Pop \leftarrow \text{NSGAIISelect}(Pop)$
for $gen = 1$ to N_{gen} **do**
  $Parents \leftarrow \textsc{CrowdedTournament}(Pop)$
  $Offspring \leftarrow \textsc{Clone}(Parents)$
  for all pairs (c_1, c_2) in $Offspring$ **do**
    if $rand < p_{\text{cx}}$ **then**
      $(c_1, c_2) \leftarrow \textsc{CrossoverSys}(c_1, c_2)$ ▷ Alg. 3
    end if
  end for
  for all $child \in Offspring$ **do**
    if $rand < p_{\text{mut}}$ **then**
      $child \leftarrow \textsc{MutateSys}(child)$ ▷ Alg. 4
    end if
    $child \leftarrow \textsc{MutateTimeBudget}(child)$ ▷ Alg. 9
    $child$.fit $\leftarrow \textsc{Evaluate}(child)$ ▷ Alg. 2
  end for
  $Pop \leftarrow \text{NSGAIISelect}(Pop \cup Offspring)$
end for
return $\arg\min_{ind \in Pop} ind$.fit

(ii) a system-level partition order, (iii) deterministic route selections for inter-partition messages, and (iv) per-partition time budgets. The system-level objective is to achieve end-to-end schedulability by minimizing the global makespan of the application while satisfying the global deadline and the per-partition budget constraints. Infeasible solutions, including those with partition budget overruns or missing feasible deterministic routes for inter-partition transfers, are penalized to steer the search toward feasible schedules.

The exploration covers trade-offs between computation and communication across tiers. For example, allocating a partition to a higher tier may reduce its processing time while increasing the latency of its inter-partition communications, whereas allocating it closer to the edge may reduce communication delay at the cost of longer execution times. Deterministic route selections are enforced by assigning each inter-partition message to a specific candidate path from the path catalog, thereby avoiding runtime route jitter. The resulting system-level schedule specifies where partitions execute, in what order they are integrated, how inter-partition messages are routed over deterministic paths, and which time budgets constrain the partition-level schedulers. The system-level scheduling procedure is summarized in Algorithm 1.

Each system-level individual is evaluated by invoking the partition-level scheduler for every partition and reconstructing an end-to-end schedule to compute feasibility violations and the global makespan. This evaluation procedure is summarized in Algorithm 2.

The system-level search relies on genetic operators that preserve the validity of the permutation-based genome components and maintain the alignment between the partition order and the time-budget vector. The crossover operator is defined in Algorithm 3.

Algorithm 4 defines the structure-preserving mutation operator applied to the system-level genome, with repairs to maintain feasibility under the model constraints.

Algorithm 2 EVALUATE

Input: Individual $ind = [\,\pi \mid \Phi \mid \rho \mid TB\,]$; partition data $(PT^p, \mathcal{M}^p_{\text{intra}}, JD^p)$
inter-partition messages $\mathcal{M}_{\text{inter}}$; path catalogs $\{\mathcal{P}_{s,r}\}$; global deadline D
Output: Fitness $\left(\sum_p \text{viol}[p], \max(0, MS_{\text{global}} - D)\right)$
and evaluation artifacts attached to ind

Build $triple[p] = (\text{alloc}{=}\Phi[i], \text{tb}{=}TB[i])$ for each partition $p = \pi[i]$
for all partitions p in order π **do**
  $(sched^p, ms^p, \phi_p) \leftarrow$ PARTITIONLEVELSCHEDULER $(p, PT^p, \mathcal{M}^p_{\text{intra}}, JD^p, triple[p], \{\mathcal{P}_{s,r}\})$
  $results[p] \leftarrow sched^p$; $maps[p] \leftarrow \phi_p$; $makespans[p] \leftarrow ms^p$
  $\text{viol}[p] \leftarrow \max(0, ms^p - triple[p].\text{tb})$
end for
$(globalSched, MS_{\text{global}}, IPCviol) \leftarrow$ SYSTEMLEVELRECONSTRUCTION $(results, maps, \mathcal{M}_{\text{inter}}, \rho, \{\mathcal{P}_{s,r}\})$
$latenessSigned \leftarrow MS_{\text{global}} - D$
$latenessClipped \leftarrow \max(0, latenessSigned)$
Attach to ind: $eval\_\pi, eval\_\Phi, eval\_\rho, eval\_TB, results, maps, makespans, \text{viol}$ $MS_{\text{global}}, latenessSigned, IPCviol$
return $\left(\sum_p \text{viol}[p], latenessClipped\right)$

Algorithm 3 CROSSOVERSYS

Input: Parent chromosomes c_1, c_2 with genome $[\,\pi \mid \Phi \mid \rho \mid TB\,]$
Output: Offspring (c_1', c_2') with valid π, Φ and aligned TB

Apply PMX on π to obtain valid partition-order permutations
Apply PMX with repair on Φ to preserve a valid allocation structure
Realign TB to the new π so that budgets remain attached to partition IDs
Apply two-point crossover on ρ
return updated (c_1, c_2)

Algorithm 4 MUTATESYS

Input: Individual $child = [\,\pi \mid \Phi \mid \rho \mid TB\,]$
Output: Mutated individual with valid π, Φ and aligned TB

Perform a synchronized swap on (π, Φ, TB) at two positions
Optionally swap two entries in Φ as an allocation tweak
Repair π and Φ if validity constraints are violated
return *child*

Deterministic routing decisions are realized by selecting a concrete path for each inter-partition message from its candidate path set. The path selection function used during evaluation and reconstruction is given in Algorithm 5.

Algorithm 5 SELECTPATH

Input: Route-index hint h; candidate paths $\mathcal{P}_{s,r}$ for endpoints (s, r)
Output: Chosen path $p^\star$

$C \leftarrow \mathcal{P}_{s,r}$
if $C = \varnothing$ **then**
  return penalized virtual path
end if
if $0 \le h < |C|$ **then**
  return $C[h]$
else
  return $\arg\min_{p \in C} \text{latency}(p)$
end if

Algorithm 6 SYSTEMLEVELRECONSTRUCTION

Input: Per-partition schedules $\{sched^p\}$ with task times and task mappings $\{\phi_p\}$
inter-partition messages $\mathcal{M}_{\text{inter}}$; route-index hints ρ; path catalogs $\{\mathcal{P}_{s,r}\}$
Output: $globalSched$ containing tasks and inter-partition transmissions; MS_{global}; set $IPCviol$

Assemble $globalSched$ from $\{sched^p\}$
$globalSched$.tasks $\leftarrow \bigcup_p sched^p$
$globalSched$.transmissions $\leftarrow \varnothing$
$M \leftarrow \mathcal{M}_{\text{inter}}$; $IPCviol \leftarrow \varnothing$
Initialize arrival constraints:
for each task v, set ArriveAfter$[v] \leftarrow start(v)$
for all $m_{u,v} \in M$ **do**
  $s \leftarrow \phi_{\text{Part}(u)}(u)$; $r \leftarrow \phi_{\text{Part}(v)}(v)$
  $C \leftarrow \mathcal{P}_{s,r}$
  if $C = \varnothing$ **then**
    $IPCviol \leftarrow IPCviol \cup \{m_{u,v}\}$
    $arr(m_{u,v}) \leftarrow +\infty$
  else
    $idx \leftarrow \rho[m_{u,v}]$
    if $0 \le idx < |C|$ **then**
      $path(m_{u,v}) \leftarrow C[idx]$
    else
      $path(m_{u,v}) \leftarrow \arg\min_{p \in C} \text{latency}(p)$
    end if
    $arr(m_{u,v}) \leftarrow end(u) + \text{latency}\big(path(m_{u,v}), m_{u,v}\big)$
    Add $(m_{u,v}, tx_s{=}end(u), tx_e{=}arr(m_{u,v}), path(m_{u,v}))$ to $globalSched$.transmissions
    ArriveAfter$[v] \leftarrow \max\big(\text{ArriveAfter}[v], arr(m_{u,v})\big)$
  end if
end for
ENFORCEARRIVALCONSISTENCY $\big(globalSched.\text{tasks}, \text{ArriveAfter}\big)$
ALIGNPROCESSORQUEUES $\big(globalSched.\text{tasks}\big)$
$MS_{\text{global}} \leftarrow$ maximum end-time over all tasks in $globalSched$.tasks
return $globalSched$, MS_{global}, $IPCviol$

4.3. End-to-End Reconstruction

This subsection describes the reconstruction step that integrates partition schedules and inserts inter-partition transmissions under deterministic routing. Algorithm 6 summarizes this procedure.

For each inter-partition message $m_{u,v} \in \mathcal{M}_{inter}$, the sender and receiver End Systems are identified from the task mappings, and a feasible candidate path is selected using the route-index hint $\rho[m_{u,v}]$. If the hinted index is invalid, the path with minimum latency is selected. The message arrival time is computed as $end(u)$ plus the bounded latency of the selected path, and the consumer task is constrained to start no earlier than this arrival time. Messages for which no feasible deterministic path exists are recorded as inter-partition communication violations. The global makespan MS_{global} is computed as the latest completion time over all tasks after enforcing these arrival constraints.

Here, $IPCviol$ denotes the set of inter-partition communication violations, i.e., messages in $\mathcal{M}_{inter}$ for which no feasible deterministic path exists.

4.4. Partition-Level Scheduler

Once the system-level scheduler fixes the allocation $\Phi(p)$ and time budget $TB(p)$ for a partition p, a local optimization is performed to construct a feasible schedule within the assigned execution domain. Each partition-level scheduler operates independently and can be executed in parallel for different partitions. Its role is to determine a precedence-feasible task order, a task-to-End-System mapping within the allocated domain, and the corresponding intra-partition communication decisions while respecting the local time budget.

For each partition p, the local optimization problem is to minimize the partition makespan subject to the assigned scheduling time-budget constraint:

$$\min\ (ms^p) \qquad \text{s.t.} \qquad ms^p \le TB(p). \tag{6}$$

ms^p denotes the completion time of the last task in partition p, whereas $TB(p)$ is the time budget assigned by the system-level scheduler. Schedules that exceed the assigned budget are treated as locally infeasible and contribute a violation term during the system-level evaluation.

To solve this problem, a genetic search is applied at the partition level. Each local individual is encoded as

$$g^p = [\,\sigma^p \mid \phi_p \mid \eta^p \mid \xi^p\,], \tag{7}$$

where σ^p is a precedence-feasible task order, ϕ_p is the mapping of tasks to End Systems within the allocated domain $\Phi(p)$, η^p defines local message priorities, and ξ^p stores the candidate-path indices used for intra-partition communications. The fitness of a local individual is defined as

$$fit(g^p) = (ms^p,\ \max(0, ms^p - TB(p))), \tag{8}$$

which simultaneously favors short makespans and penalizes budget overruns.

Algorithm 7 summarizes the partition-level scheduler. The partition-level search begins by generating an initial population of valid local individuals. The task-order component σ^p is initialized using random topological orders derived from the precedence relations in JD^p, thereby ensuring that all local individuals satisfy intra-partition dependencies from the outset. The mapping component ϕ_p assigns tasks only to End Systems belonging to the execution domain selected by $\Phi(p)$, while η^p and ξ^p initialize message priorities and local path selections for intra-partition communications.

Algorithm 7 PARTITIONLEVELSCHEDULER

Input: Partition p; data $(PT^p, \mathcal{M}^p_{\text{intra}}, JD^p)$; (alloc=$\Phi(p)$, tb=$TB(p)$)
path catalogs $\{\mathcal{P}_{s,r}\}$; inner GA parameters $(N^{(p)}_{\text{pop}}, N^{(p)}_{\text{gen}}, p^{(p)}_{\text{cx}}, p^{(p)}_{\text{mut}})$
Output: Local schedule $sched^p$, local makespan ms^p, and task mapping ϕ_p

Genome: $g^p = [\,\sigma^p \mid \phi_p \mid \eta^p \mid \xi^p\,]$
$Pop^p \leftarrow$ INITLOCALPOPULATION$(p, \Phi(p))$
$best^p \leftarrow \varnothing$; $bestMS^p \leftarrow +\infty$
for $gen = 1$ to $N^{(p)}_{\text{gen}}$ **do**
  for all $g^p \in Pop^p$ **do**
    $(sched^p_g, ms^p_g, \phi^g_p) \leftarrow$ RECONSTRUCTPARTITION $(JD^p, g^p, PT^p, \mathcal{M}^p_{\text{intra}}, \{\mathcal{P}_{s,r}\})$ ▷ Alg. 8
    $fit(g^p) \leftarrow (ms^p_g, \max(0, ms^p_g - TB(p)))$
    if $ms^p_g < bestMS^p$ **then**
      $best^p \leftarrow g^p$; $bestMS^p \leftarrow ms^p_g$
    end if
  end for
  $Parents^p \leftarrow$ TOURNAMENTSELECT(Pop^p, fit)
  $Offspring^p \leftarrow$ CROSSOVERLOCAL$(Parents^p)$
  $Offspring^p \leftarrow$ MUTATELOCAL$(Offspring^p)$
  $Pop^p \leftarrow$ SELECTNEXTLOCAL$(Parents^p \cup Offspring^p, N^{(p)}_{\text{pop}})$
end for
$(sched^p, ms^p, \phi_p) \leftarrow$ RECONSTRUCTPARTITION $(JD^p, best^p, PT^p, \mathcal{M}^p_{\text{intra}}, \{\mathcal{P}_{s,r}\})$
return $(sched^p, ms^p, \phi_p)$

At each generation, every local individual is evaluated through a deterministic reconstruction step that converts the encoded order, mapping, and communication choices into a concrete schedule. The resulting local makespan ms^p is then combined with the

budget-overrun term $\max(0, ms^p - TB(p))$ to compute the local fitness according to (8). The evolutionary process then applies selection, crossover, and mutation operators that preserve precedence feasibility and allocation validity. Once the local search terminates, the best reconstructed schedule is returned to the system-level scheduler, together with its realized task mapping and makespan.

For a fixed genome, the reconstruction procedure deterministically produces a unique local schedule. The start time of each task is computed solely from predecessor completion times, communication latencies along the selected paths, and the availability of the assigned End System. Consequently, the resulting makespan and any budget violation depend only on the decisions encoded in the genome. This determinism ensures consistent evaluation of individuals at the system level and allows scheduling decisions to be compared reliably during the evolutionary search.

4.5. Local Schedule Reconstruction

Given a local genome $g^p = [\,\sigma^p \mid \phi_p \mid \eta^p \mid \xi^p\,]$, the reconstruction procedure deterministically constructs the schedule of partition p by assigning start and completion times to all tasks and by inserting the corresponding intra-partition transmissions.

The reconstruction must satisfy three conditions:(i) precedence feasibility between tasks, (ii) exclusive use of each End System, and (iii) causal consistency of intra-partition communications.

Tasks are processed according to the precedence-feasible order encoded in σ^p. For each task t, the earliest feasible start time is computed as the maximum between processor availability and the arrival time of all incoming messages. The assigned End System is determined by $\phi_p(t)$, while the communication latency of each incoming message is derived from the candidate path selected through the path-index vector ξ^p.

Once a task is scheduled, all outgoing intra-partition messages are inserted according to the message-priority vector η^p. The resulting message transmission times are then propagated to successor tasks to ensure that no task starts before all required data has arrived.

The partition makespan ms^p is finally obtained as the maximum completion time among all tasks in the reconstructed schedule. Algorithm 8 summarizes this procedure.

Algorithm 8 RECONSTRUCTPARTITION (Local Reconstruction)

Input: JD^p; genome $g^p = \langle \sigma^p, \phi_p, \eta^p, \xi^p \rangle$; PT^p; $\mathcal{M}^p_{\text{intra}}$; path catalogs $\{\mathcal{P}_{s,r}\}$
Output: $sched^p$ containing task and intra-partition message times; local makespan ms^p

Initialize $next\_free[q] \leftarrow 0$ for every end system q used in ϕ_p
for all task t in σ^p **do**
  $q \leftarrow \phi_p[t]$
  predReady $\leftarrow \max_{u \in Pred(t)} (end(u) + \text{latency}(path_{u,t}, m_{u,t}))$ where $path_{u,t} = \text{SELECTPATH}(\xi^p[m_{u,t}], \mathcal{P}_{\phi_p(u),q})$ ▷ 0 if $Pred(t) = \varnothing$
  procReady $\leftarrow next\_free[q]$
  $start(t) \leftarrow \max(\text{predReady}, \text{procReady})$
  $end(t) \leftarrow start(t) + PT^p(t, q)$
  $next\_free[q] \leftarrow end(t)$
  for all intra-partition messages $m : t \to v$ in $\mathcal{M}^p_{\text{intra}}$ **do** ordered by $\eta^p[m]$
    $q_v \leftarrow \phi_p[v]$
    $path \leftarrow \text{SELECTPATH}(\xi^p[m], \mathcal{P}_{q,q_v})$ ▷ Alg. 5
    $tx\_s(m) \leftarrow end(t)$
    $tx\_e(m) \leftarrow tx\_s(m) + \text{latency}(path, m)$
    enforce $start(v) \leftarrow \max(start(v), tx\_e(m))$ ▷ propagate to successors if needed
  end for
end for
$ms^p \leftarrow \max_t end(t)$
return $sched^p, ms^p$

4.6. Time-Budget Mutation and Slack Distribution

A key innovation in the system-level GA is the time-budget mutation operator described in Algorithm 9, which dynamically adjusts the partition time budgets to improve convergence. Instead of applying blind mutation to the time-budget vector, the operator redistributes slack among partitions based on the feedback obtained from the previous evaluation.

During mutation, partitions that finish significantly earlier than their assigned budget act as *donors*, whereas partitions whose execution time approaches or exceeds their allocated budget act as *receivers*. A portion of the slack from donor partitions is transferred to receiver partitions by tightening the donor budgets and relaxing the receiver budgets accordingly. This redistribution preserves the overall temporal structure of the schedule while shifting idle time toward partitions that require additional execution margin.

By continuously balancing slack among partitions, the system-level search can escape stagnation caused by poor initial budget assignments. Over successive generations, the mutation operator progressively stabilizes the budget distribution so that partitions neither suffer from excessive deadline pressure nor accumulate unnecessary slack. Empirically, this results in faster convergence toward feasible end-to-end schedules because the evolutionary search avoids exploring obviously infeasible budget allocations.

Algorithm 9 MUTATETIMEBUDGET

Input: Individual $ind = [\,\pi \mid \Phi \mid \rho \mid TB\,]$ with stored partition makespans $makespans[p]$ and violations $\mathrm{viol}[p]$
Output: Updated TB aligned with permutation π

Parameters: margin ratios, stability-band limits, step gains, and bounded perturbation factor
for $i = 0$ to $|\pi| - 1$ **do**
  $p \leftarrow \pi[i]$
  $tb \leftarrow TB[i]$
  $ms \leftarrow makespans[p]$
  $v \leftarrow \mathrm{viol}[p]$
  $target \leftarrow ms(1+\delta)$ ▷ safety margin
  Define stability band $(band_{\mathrm{lo}}, band_{\mathrm{hi}})$ around $target$
  if $v > 0$ **or** $tb < ms$ **then** ▷ deficit
    Increase $TB[i]$ proportionally toward $target$
  else if $tb \geq band_{\mathrm{hi}}$ **then** ▷ surplus
    Decrease $TB[i]$ proportionally toward $band_{\mathrm{lo}}$
  else if $tb < target$ **then**
    Slightly increase $TB[i]$ toward $band_{\mathrm{lo}}$
  else
    Keep $TB[i]$ unchanged
  end if
  Optionally apply a small bounded perturbation
end for
return TB

5. Performance Evaluation

This section evaluates the proposed hierarchical genetic algorithm (HGA) with respect to end-to-end schedulability, global makespan, and convergence behavior. The evaluation is organized into two parts. First, the proposed method is benchmarked against established scheduling baselines under the baseline deadlines of the considered workloads. Second, a sensitivity analysis examines the behavior of the adaptive slack reallocation mechanism under progressively tightened deadlines.

5.1. Evaluation Setup

The proposed scheduler was implemented in Python 3.10 using the DEAP evolutionary computation framework [48]. The evaluation used synthetic directed acyclic graph (DAG) workloads with 100, 250, and 500 tasks. The use of synthetic DAGs is well established in scheduling research, where generators such as TGFF [49] and SDF3 [50] are widely used to enable controlled and repeatable benchmarking across diverse application structures. Following this methodology, the DAGs in this work were generated using the Stanford Network Analysis Platform (SNAP), which supports systematic variation in graph size and structural properties, including dependency density and in- and out-degree distributions. This enables reproducible evaluation across heterogeneous workload patterns while preserving comparability with prior DAG-based scheduling studies.

These workloads were evaluated on a fixed edge–fog–cloud platform model represented as a graph of compute and communication resources. The compute infrastructure comprises one edge–fog domain (FE) and three cloud clusters (C1, C2, and C3). For the communication infrastructure, a finite set of deterministic candidate routes was computed offline on the platform graph using a K-shortest-path method for each relevant source–destination pair. During scheduling, routing decisions were then made by selecting from this precomputed path catalog. The same platform model was used across all workload sizes and deadline settings, so the observed performance differences are attributable to scheduling decisions rather than changes in the underlying execution infrastructure.

Within this model, application DAG vertices represent tasks and DAG edges represent inter-task messages. Tasks are associated with execution-time parameters, such as worst-case execution times, while messages incur deterministic communication costs once routing paths are selected from the precomputed path catalog. This allows both computation placement and communication-path selection to be evaluated consistently within the same scheduling framework.

The baseline application deadlines were derived from the makespan of an initial list-scheduling heuristic applied to each workload and correspond to the 100% deadline setting used in the main benchmark comparison. To establish a challenging theoretical lower bound, this initial heuristic calculates a baseline makespan by greedily assigning topologically sorted tasks to processors to minimize early finish times. The baseline deadlines were set to 2600, 2700, and 4300 TU for the 100T, 250T, and 500T workloads, respectively. Additional experiments with tightened deadlines of 90%, 80%, and 70% of these baseline values are presented later in the sensitivity analysis to evaluate the robustness of the proposed approach under reduced timing slack.

At the system level, the optimization jointly minimizes total partition budget violations and global lateness, where lateness is defined as the difference between the global makespan and the application deadline. At the partition level, the local scheduler minimizes partition makespan while penalizing partition budget overruns. This objective hierarchy aligns with the hierarchical feasibility model, in which partition-level schedules are evaluated against local time-budget constraints within the end-to-end reconstruction process.

Tables 2 and 3 summarize the parameters used for the system-level and partition-level genetic algorithms, respectively.

Table 2. System-level GA parameters.

Parameter	Value
Population size	16
Generations	60
Crossover probability	0.80
Mutation probability	0.70
Fitness objective 1	Cumulative partition budget violations
Fitness objective 2	Positive global lateness
Selection strategy	NSGA-II

Table 3. Partition-level GA parameters.

Parameter	Value
Population size	16
Generations	20
Crossover probability	0.70
Mutation probability	0.70
Fitness objective 1	Partition makespan
Fitness objective 2	Partition budget overruns

HEFT and Round-Robin are used as conventional external reference schedulers for heterogeneous task scheduling. Since they do not include hierarchical partition-level time budgets, feedback-guided budget adaptation, or route-selection genes, they are not used to isolate the contribution of individual HGA mechanisms. The controlled variants in Section 5.6 specifically isolate the contribution of adaptive time-budget reallocation while retaining the remaining hierarchical scheduling structure.

5.2. Compared Schedulers

In this subsection, we compare our proposed HGA against HEFT and Round-Robin schedulers.

Heterogeneous Earliest-Finish-Time (HEFT)

HEFT [19] prioritizes tasks according to upward rank and assigns each task to the processing element that minimizes its earliest finish time. It is precedence-aware and communication-aware at the cost-model level. However, it does not perform hierarchical system-partition co-design, adaptive time-budget reallocation, or explicit deterministic path selection. To apply HEFT within our architecture, we restricted it to static, predefined partition-to-tier assignments, as its standard architecture lacks the hierarchical mechanisms required to allocate entire partitions across multi-tier architectures dynamically. Furthermore, while HEFT evaluated communication using the same bounded $L(m, p)$ costs as the proposed framework, it does not possess the native capability to evaluate and select alternative paths from the deterministic catalog dynamically. Because path selection falls outside HEFT's optimization space, it evaluated communication costs using a static, baseline path assignment from the deterministic catalog. Consequently, unlike the proposed HGA, HEFT cannot dynamically search the catalog to secure the lowest-latency route.

Round-Robin (RR)

Round-Robin dispatches ready tasks cyclically according to a fixed service order. Although simple and robust, it does not exploit critical-path structure, does not coordinate decisions across partitions, and does not adapt time budgets based on feasibility feedback.

Neither baseline jointly optimizes partition allocation, partition execution order, deterministic routing, and partition time budgets. Consequently, they do not provide the same end-to-end co-design capability as the proposed HGA.

5.3. Benchmark Results Under Baseline Deadlines

Table 4 summarizes the final results obtained for the three workload sizes under the baseline deadline setting.

Table 4. Global schedulability results under the baseline deadline setting; all values are in TU.

Workload	Deadline	HGA Makespan	Global Lateness	Violation Sum
100T	2600	2317	0	0
250T	2700	2229	0	0
500T	4300	4021	0	0

The proposed HGA satisfies the application deadlines in all three evaluated workloads while simultaneously eliminating partition budget violations. This confirms that the hierarchical scheduler is able to construct fully feasible schedules under the considered deterministic execution and communication model.

In addition to satisfying feasibility, the obtained makespans remain below the corresponding deadlines. The resulting slack margins are 283 TU for 100T, 471 TU for 250T, and 279 TU for 500T, indicating that the scheduler does not merely converge to deadline-feasible solutions but also preserves timing headroom.

To further examine the behavior of the adaptive slack reallocation mechanism, Table 5 reports the final partition budgets and realized partition makespans under the baseline experiments.

Table 5. Partition budgets and realized partition makespans at convergence under baseline deadlines; all values are in TU.

Workload	FE		C1		C2		C3	
	Budget	MS	Budget	MS	Budget	MS	Budget	MS
100T	1995	1994	148	129	253	244	972	943
250T	1117	1111	408	345	827	796	448	399
500T	2275	2251	910	867	433	410	443	419

As shown in Table 5, the partition time budgets converge toward the corresponding realized partition makespans across the evaluated workloads. The adaptive slack reallocation mechanism progressively adjusts each partition budget based on the execution behavior returned by the partition-level scheduler. As a result, the remaining gap between assigned budgets and realized makespans is reduced across partitions, indicating reduced under-provisioning and over-provisioning at convergence.

Table 6 compares the proposed HGA against HEFT and Round-Robin under the same baseline deadline setting.

Table 6. Comparison of global makespan across scheduling methods.

Workload	HGA	HEFT	RR	Deadline
100T	2317	4329	2714	2600
250T	2229	3848	2698	2700
500T	4021	4726	4958	4300

The proposed HGA achieves the lowest global makespan in all evaluated cases. Relative to HEFT, the makespan reductions are 46.5% for 100T, 42.1% for 250T, and 14.9% for 500T. Relative to Round-Robin, the reductions are 14.6%, 17.4%, and 18.9%, respectively. Moreover, while the proposed HGA satisfies all baseline deadlines, HEFT exceeds the deadline in all three workloads and Round-Robin exceeds the deadline in the 100T and 500T workloads.

The observed makespan differences are consistent with the broader decision space available to the proposed HGA, which jointly searches partition allocation, partition execution order, route-selection hints, and partition time budgets. However, the external baseline comparison does not by itself attribute the improvement to any single mechanism.

5.4. Convergence and Adaptive Budget Behavior

Table 7 reports the evolution of the cumulative partition violation sum and global lateness across the evaluated scenarios. In all cases, the partition violation sum converges to zero, indicating that partition-level feasibility is eventually achieved. The table also shows whether this local feasibility is accompanied by zero final global lateness or by a residual end-to-end timing gap at convergence.

Table 7. Convergence summary across evaluated scenarios.

Workload Size	Deadline Setting	Initial Viol.	Final Viol.	First Gen. Zero Viol.	Initial Late.	Final Late.
100T	100%	1544	0	25	0	0
100T	90%	1688	0	27	149	0
100T	80%	1755	0	28	117	91
100T	70%	1960	0	31	650	488
250T	100%	520	0	10	0	0
250T	90%	649	0	12	0	0
250T	80%	725	0	15	0	0
250T	70%	929	0	15	36	0
500T	100%	1215	0	24	0	0
500T	90%	1239	0	23	65	0
500T	80%	1292	0	30	1026	115
500T	70%	1514	0	31	976	607

With respect to the end-to-end deadline, the results show a more selective behavior. Zero global lateness is achieved in all baseline cases and in several tightened-deadline scenarios, namely 100T at 90%, 250T at 90%, 80%, and 70%, and 500T at 90%. In contrast, the 100T–80%, 100T–70%, 500T–80%, and 500T–70% cases converge with residual lateness values of 91, 488, 115, and 607 TU, respectively. These cases show that local partition feasibility does not necessarily imply end-to-end feasibility when the imposed deadline becomes too restrictive. Instead, the search converges to a stable compromise solution under the available computation and communication resources.

Table 8. Reduction in total budget–makespan gap across partitions.

Workload Size	Deadline Setting	Initial $\sum_p \lvert TB_p - MS_p \rvert$	Final $\sum_p \lvert TB_p - MS_p \rvert$	Reduction (%)
100T	100%	2394	58	97.6
100T	90%	2416	54	97.8
100T	80%	2421	79	96.7
100T	70%	2441	79	96.8
250T	100%	1044	149	85.7
250T	90%	1047	73	93.0
250T	80%	999	85	91.5
250T	70%	1086	61	94.4
500T	100%	2581	114	95.6
500T	90%	2304	119	94.8
500T	80%	2079	166	92.0
500T	70%	2096	150	92.8

To quantify the adaptive budget behavior directly, Table 8 reports the aggregated budget–makespan mismatch, measured as $\sum_p |TB_p - MS_p|$ across all partitions. This quantity decreases substantially in every evaluated scenario, with reductions ranging from 85.7% to 97.8%. The reduction confirms that the adaptive slack reallocation mechanism progressively aligns the system-level time budgets with the realized execution demand returned by the partition-level schedulers. In other words, the mutation operator not only repairs violating partitions, but also reduces both over-provisioning and under-provisioning, thereby strengthening the temporal alignment between the system level and the partition level.

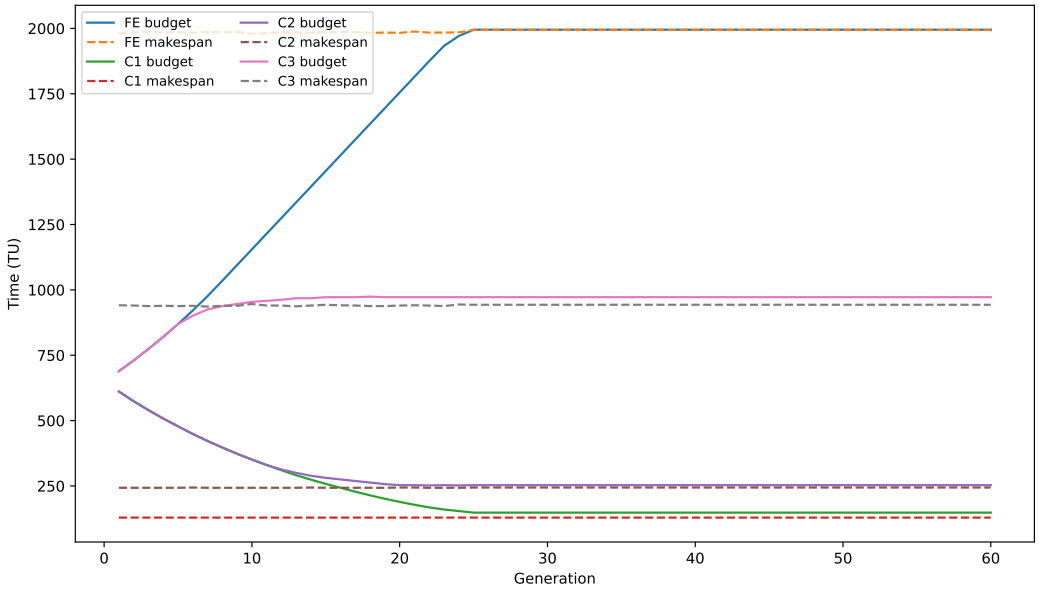

Figure 2. Generation-wise evolution of partition budgets and realized partition makespans for the 100T workload under the baseline deadline setting. The progressive alignment between the assigned budgets and the partition makespans indicates convergence of the partition-level temporal budgets.

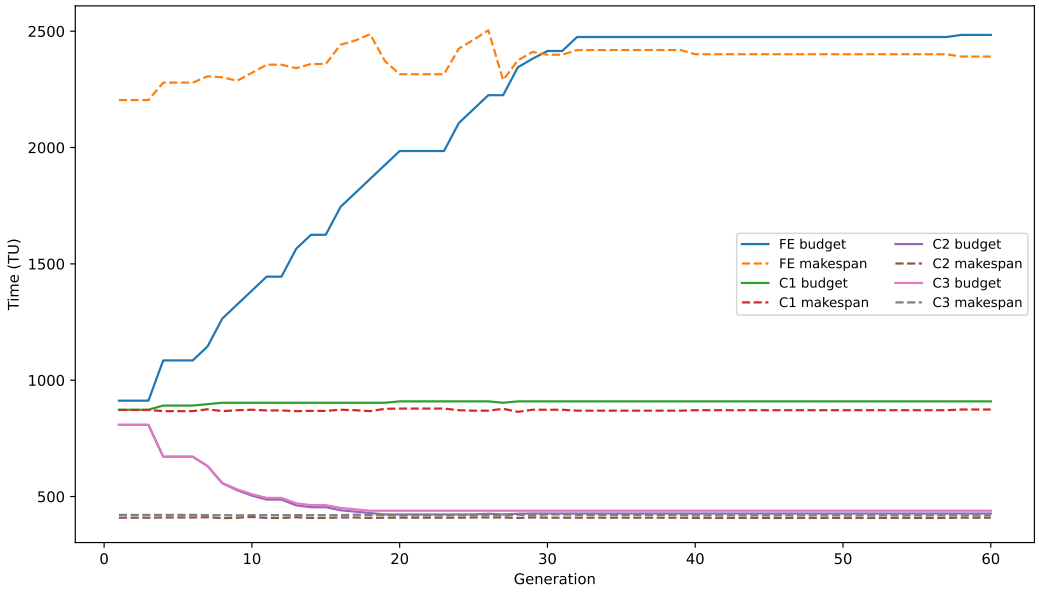

Figure 3. Generation-wise evolution of partition budgets and realized partition makespans for the 500T workload under the 80% tightened deadline setting. Although a residual global lateness remains in this case, the partition budgets still converge toward the realized partition makespans, demonstrating stable adaptive budget alignment.

Figures 2 and 3 provide a generation-wise view of this adaptation process for representative baseline and tightened-deadline cases. In both cases, the assigned partition budgets progressively move toward the realized partition makespans returned by the partition-level schedulers. For the baseline 100T case, this alignment coincides with zero final global lateness. For the tightened 500T–80% case, the same budget-alignment behavior is observed even though a residual global lateness remains. This confirms that the adaptive mutation operator stabilizes partition-level budget assignments even in scenarios where the end-to-end deadline is too restrictive to be fully satisfied.

The results show that the adaptive slack reallocation mechanism serves two roles simultaneously. First, it acts as a feasibility-repair operator that eliminates cumulative partition budget violations. Second, it acts as a convergence aid by steering partition budgets toward stable values that closely match the realized partition makespans. This behavior explains why the proposed HGA maintains robust convergence characteristics even as the global deadline is progressively tightened.

5.5. Sensitivity to Deadline Tightening

The results further indicate that the proposed HGA is sensitive to progressively tightened end-to-end deadlines. As the deadline is reduced from the baseline setting to 90%, 80%, and 70%, the feasible region of the search space shrinks, and satisfying both partition-level feasibility and global end-to-end feasibility becomes increasingly challenging.

For moderate tightening, the proposed approach remains robust. Several 90% cases and all evaluated 250T cases still achieve zero final global lateness, indicating that the adaptive time-budget negotiation mechanism can absorb a substantial reduction in the global timing margin while preserving end-to-end schedulability. This behavior suggests that, when sufficient structural slack is still available across partitions and communication paths, the HGA can successfully redistribute time budgets and recover a globally feasible solution.

Under stronger tightening, however, the feasible region shrinks significantly. In particular, the 100T–80%, 100T–70%, 500T–80%, and 500T–70% scenarios converge with nonzero residual lateness. This behavior indicates that the algorithm is no longer limited by poor budget assignment alone, but by the reduced global temporal margin imposed by the end-to-end deadline itself. In such cases, the adaptive mutation operator still eliminates partition-level violations and stabilizes the partition budgets, but the remaining timing gap cannot be fully removed without relaxing the deadline or altering the available compute and communication resources.

An important observation is that the performance degradation is gradual rather than abrupt as the deadline is tightened. Even in scenarios where a nonzero residual lateness remains, the search continues to reduce both the violation term and the budget–makespan mismatch, converging to stable compromise solutions. This indicates that the algorithm does not exhibit oscillatory or unstable behavior, but instead maintains consistent convergence toward the best achievable solution under the given constraints. Such behavior is particularly important for hierarchical scheduling, as it shows that the proposed HGA remains stable and effective even when full end-to-end feasibility cannot be achieved.

The deadline-tightening study shows that the proposed method remains effective as the timing constraints become stricter. When a feasible schedule exists, the HGA is able to find it. When the deadline is too tight to be fully satisfied, the algorithm still converges to stable solutions with zero partition-level violations and lower global lateness. This shows that the hierarchical search remains robust and that the adaptive time-budget negotiation mechanism continues to improve the solution even under tight timing constraints.

5.6. Ablation Study of Adaptive Time-Budget Reallocation

This subsection isolates the contribution of adaptive time-budget reallocation. The ablation study is conducted on the 100T and 250T workloads under the deadline settings introduced earlier, namely 70%, 80%, 90%, and 100% of the baseline deadline. These correspond to $D = \{1820, 2080, 2340, 2600\}$ TU for 100T and $D = \{1890, 2160, 2430, 2700\}$ TU for 250T. For each workload–deadline setting, each variant is executed with 10 independent random seeds.

Table 9. Scheduler variants used in the ablation study.

Variant	TB Genes	TB Mutation	Partition Feedback
Proposed-HGA	Yes	Yes	Yes
HGA-NoFeedback	Yes	Yes	No
HGA-StaticBudget	Fixed	No	No

The ablation study provides the controlled mechanism-level comparison for the proposed HGA. Unlike HEFT and Round-Robin, the ablation variants preserve the same hierarchical scheduling structure and deterministic path selection as the proposed method. Three scheduler variants are compared, as summarized in Table 9. *Proposed-HGA* denotes the complete method, where partition time budgets are adapted using the proposed slack reallocation technique. This technique uses feedback from the observed partition makespans, budget violations, and available slack to adjust the time-budget distribution during the system-level search. *HGA-NoFeedback* keeps the same hierarchical GA structure and keeps mutable time-budget genes, but replaces feedback-guided budget adaptation with bounded random time-budget mutation. Meaning that time budgets can still evolve, but their changes are not guided by partition slack or violation feedback. *HGA-StaticBudget* fixes the partition time budgets after initialization using a workload-aware static rule. The comparison therefore separates three effects: fixed time budgets, mutable but feedback-free time budgets, and feedback-guided adaptive time-budget reallocation.

Table 10. Feasibility and partition-budget convergence results for the ablation study over 10 independent runs. The first feasible generation is computed over the runs in which all partition time budgets were satisfied.

Workload	Scheduler variant	Deadline value (TU)	Runs	Deadline met (%)	Budgets met (%)	Fully feasible (%)	First feasible generation
100T	Proposed-HGA	1820	10	0.0	100.0	0.0	31.0 ± 0.0
100T	HGA-NoFeedback	1820	10	0.0	0.0	0.0	–
100T	HGA-StaticBudget	1820	10	0.0	0.0	0.0	–
100T	Proposed-HGA	2080	10	0.0	100.0	0.0	29.0 ± 0.0
100T	HGA-NoFeedback	2080	10	0.0	40.0	0.0	59.0 ± 1.4
100T	HGA-StaticBudget	2080	10	0.0	0.0	0.0	–
100T	Proposed-HGA	2340	10	100.0	100.0	100.0	26.9 ± 0.3
100T	HGA-NoFeedback	2340	10	100.0	100.0	100.0	54.0 ± 2.5
100T	HGA-StaticBudget	2340	10	30.0	0.0	0.0	–
100T	Proposed-HGA	2600	10	100.0	100.0	100.0	25.0 ± 0.0
100T	HGA-NoFeedback	2600	10	100.0	100.0	100.0	45.0 ± 2.4
100T	HGA-StaticBudget	2600	10	90.0	0.0	0.0	–
250T	Proposed-HGA	1890	10	100.0	100.0	100.0	16.6 ± 1.3
250T	HGA-NoFeedback	1890	10	50.0	100.0	50.0	50.0 ± 2.8
250T	HGA-StaticBudget	1890	10	0.0	0.0	0.0	–
250T	Proposed-HGA	2160	10	100.0	100.0	100.0	13.4 ± 0.9
250T	HGA-NoFeedback	2160	10	100.0	100.0	100.0	35.5 ± 3.7
250T	HGA-StaticBudget	2160	10	50.0	0.0	0.0	–
250T	Proposed-HGA	2430	10	100.0	100.0	100.0	11.3 ± 0.8
250T	HGA-NoFeedback	2430	10	100.0	100.0	100.0	27.0 ± 3.5
250T	HGA-StaticBudget	2430	10	100.0	0.0	0.0	–
250T	Proposed-HGA	2700	10	100.0	100.0	100.0	9.4 ± 0.7
250T	HGA-NoFeedback	2700	10	100.0	100.0	100.0	23.3 ± 3.1
250T	HGA-StaticBudget	2700	10	100.0	0.0	0.0	–

A run is considered deadline-feasible if $MS_{\text{global}} \leq D$. It is considered partition-time-budget feasible if

$$\sum_{p \in \mathcal{P}} \max(0, MS_p - TB_p) = 0.$$

A run is fully feasible only when both conditions are satisfied. This distinction is important because a schedule may satisfy the global end-to-end deadline while still violating one or more partition time budgets.

Table 10 reports the feasibility rates and the generation at which all partition time budgets are first satisfied. The results show that fixed workload-aware time budgets are not sufficient for hierarchical schedulability in this setting. HGA-StaticBudget meets the global deadline in some cases, especially under relaxed deadlines, but it fails to meet partition time budgets in any evaluated setting. For example, in the 250T workload at $D = 2430$ and $D = 2700$, HGA-StaticBudget meets the global deadline in 100% of the runs, but the partition time budgets are never all satisfied. This confirms that global deadline feasibility alone does not guarantee feasibility with respect to the partition-level timing constraints used by the hierarchical scheduler.

HGA-NoFeedback improves over HGA-StaticBudget because its time-budget genes remain mutable. In several relaxed cases, it reaches the same fully feasible rate as Proposed-HGA. This indicates that time-budget mutability is useful in itself. However, the comparison also shows that feedback-free mutation is less reliable under tighter timing conditions. In the 100T workload at $D = 1820$, HGA-NoFeedback does not satisfy the partition time budgets in any run, whereas Proposed-HGA satisfies them in all runs. Similarly, at $D = 2080$, HGA-NoFeedback satisfies the partition time budgets in only 40% of the runs, while Proposed-HGA reaches 100%. For the 250T workload at the tightest deadline, $D = 1890$, Proposed-HGA is fully feasible in all runs, whereas HGA-NoFeedback is fully feasible in 50% of the runs. These cases indicate that the proposed feedback mechanism is most beneficial when the timing margin is limited and budget changes must be directed toward partitions with observed timing pressure.

The convergence behavior further supports this interpretation. Whenever both Proposed-HGA and HGA-NoFeedback eventually satisfy all partition time budgets, Proposed-HGA reaches this condition earlier. For the 100T workload, the first partition-feasible generation is reached by Proposed-HGA between generations 25 and 31 on average, compared with generations 45 to 59 for HGA-NoFeedback. For the 250T workload, Proposed-HGA reaches partition-time-budget feasibility between generations 9.4 and 16.6 on average, while HGA-NoFeedback requires between generations 23.3 and 50. This suggests that slack and violation feedback serve as a convergence aid by guiding the search toward useful budget changes rather than relying solely on undirected random budget perturbations.

Table 11. Mean ± standard deviation of the final per-partition budget–makespan difference after convergence. The value is computed as $\Delta TB_p = TB_p - MS_p$, where smaller positive values indicate tighter alignment and negative values indicate partition time-budget violations.

Workload	Scheduler	Deadline D	FE ΔTB	C1 ΔTB	C2 ΔTB	C3 ΔTB
100T	Proposed-HGA	1820	29.4 ± 9.0	5.0 ± 0.0	9.3 ± 1.0	36.8 ± 4.0
100T	HGA-NoFeedback	1820	-190.8 ± 42.9	296.5 ± 115.4	204.0 ± 91.0	121.0 ± 91.2
100T	Proposed-HGA	2080	43.3 ± 12.0	5.0 ± 0.0	9.9 ± 0.9	35.3 ± 3.0
100T	HGA-NoFeedback	2080	-9.9 ± 35.1	386.1 ± 131.9	164.0 ± 116.1	194.7 ± 202.2
100T	Proposed-HGA	2340	9.7 ± 3.8	5.0 ± 0.0	9.1 ± 0.6	34.8 ± 2.5
100T	HGA-NoFeedback	2340	27.4 ± 11.7	475.4 ± 137.2	294.4 ± 110.3	119.1 ± 68.8
100T	Proposed-HGA	2600	7.3 ± 4.7	19.0 ± 0.0	9.4 ± 1.0	34.8 ± 2.3
100T	HGA-NoFeedback	2600	12.2 ± 11.1	466.4 ± 159.9	372.2 ± 110.6	150.6 ± 126.5
250T	Proposed-HGA	1890	56.8 ± 39.0	13.3 ± 1.7	28.8 ± 5.6	17.1 ± 0.9
250T	HGA-NoFeedback	1890	30.0 ± 8.5	101.0 ± 114.6	34.5 ± 43.1	112.0 ± 45.3
250T	Proposed-HGA	2160	19.6 ± 21.2	15.4 ± 1.6	26.0 ± 2.3	17.4 ± 1.3
250T	HGA-NoFeedback	2160	32.9 ± 34.0	148.9 ± 58.8	57.5 ± 88.4	124.5 ± 97.2
250T	Proposed-HGA	2430	26.8 ± 24.4	32.0 ± 5.5	25.0 ± 5.3	22.1 ± 4.5
250T	HGA-NoFeedback	2430	27.8 ± 30.5	193.4 ± 77.4	46.5 ± 72.8	105.5 ± 64.5
250T	Proposed-HGA	2700	17.7 ± 16.0	57.9 ± 6.7	27.3 ± 3.6	43.8 ± 3.5
250T	HGA-NoFeedback	2700	42.1 ± 33.4	275.4 ± 81.3	98.7 ± 52.1	270.8 ± 64.7

Table 11 provides a second view of the same behavior by reporting the final per-partition budget–makespan difference, computed as $\Delta TB_p = TB_p - MS_p$. Smaller positive values indicate tighter alignment between the assigned time budget and the realized partition makespan, while negative values indicate a partition time-budget violation. The results show that HGA-NoFeedback often reaches feasibility by leaving substantially larger unused time budgets. For example, in the 100T workload at $D = 2600$, HGA-NoFeedback leaves large positive differences in C1 and C2, whereas Proposed-HGA keeps all partition differences comparatively small. A similar pattern appears in the 250T workload at $D = 2700$, where HGA-NoFeedback leaves considerably larger positive differences in C1 and C3 than Proposed-HGA. Therefore, HGA-NoFeedback can be effective in relaxed cases, but its final budget distribution is less tightly aligned with the realized partition makespans.

The final timing statistics in Table 12 should therefore be interpreted together with the feasibility and budget-alignment results. HGA-NoFeedback can occasionally obtain a similar or even lower mean makespan in relaxed cases because its feedback-free budget mutation can still explore feasible allocations. However, makespan alone does not capture whether the partition time budgets are satisfied or whether the assigned budgets are tightly aligned with partition execution demand. Across the evaluated settings, Proposed-HGA consistently eliminates partition time-budget violations, reaches partition-time-budget feasibility earlier, and produces tighter budget–makespan alignment.

Table 12. Mean ± standard deviation of final timing metrics in the ablation study over 10 independent runs.

Workload	Scheduler	Deadline D	Final makespan	Partition time-budget violation	Positive global lateness
100T	Proposed-HGA	1820	2305.6 ± 4.9	0.0 ± 0.0	485.6 ± 4.9
100T	HGA-NoFeedback	1820	2475.5 ± 139.7	190.8 ± 42.9	655.5 ± 139.7
100T	HGA-StaticBudget	1820	2394.6 ± 112.1	1469.3 ± 1.5	574.6 ± 112.1
100T	Proposed-HGA	2080	2307.7 ± 1.5	0.0 ± 0.0	227.7 ± 1.5
100T	HGA-NoFeedback	2080	2379.3 ± 103.7	27.9 ± 38.7	299.3 ± 103.7
100T	HGA-StaticBudget	2080	2394.6 ± 112.1	1209.3 ± 1.5	314.6 ± 112.1
100T	Proposed-HGA	2340	2324.0 ± 10.3	0.0 ± 0.0	0.0 ± 0.0
100T	HGA-NoFeedback	2340	2323.2 ± 4.9	0.0 ± 0.0	0.0 ± 0.0
100T	HGA-StaticBudget	2340	2446.6 ± 136.7	948.6 ± 1.9	116.6 ± 125.9
100T	Proposed-HGA	2600	2408.6 ± 87.1	0.0 ± 0.0	0.0 ± 0.0
100T	HGA-NoFeedback	2600	2382.6 ± 53.7	0.0 ± 0.0	0.0 ± 0.0
100T	HGA-StaticBudget	2600	2430.6 ± 112.4	784.7 ± 0.5	2.4 ± 7.6
250T	Proposed-HGA	1890	1825.1 ± 46.6	0.0 ± 0.0	0.0 ± 0.0
250T	HGA-NoFeedback	1890	1857.5 ± 65.8	0.0 ± 0.0	7.0 ± 9.9
250T	HGA-StaticBudget	1890	2293.1 ± 118.7	1027.2 ± 0.8	403.1 ± 118.7
250T	Proposed-HGA	2160	2021.4 ± 99.2	0.0 ± 0.0	0.0 ± 0.0
250T	HGA-NoFeedback	2160	2038.6 ± 67.2	0.0 ± 0.0	0.0 ± 0.0
250T	HGA-StaticBudget	2160	2229.4 ± 209.3	954.6 ± 0.5	109.8 ± 174.7
250T	Proposed-HGA	2430	2161.8 ± 129.4	0.0 ± 0.0	0.0 ± 0.0
250T	HGA-NoFeedback	2430	2194.4 ± 136.9	0.0 ± 0.0	0.0 ± 0.0
250T	HGA-StaticBudget	2430	2139.1 ± 119.9	881.7 ± 0.7	0.0 ± 0.0
250T	Proposed-HGA	2700	2271.7 ± 242.0	0.0 ± 0.0	0.0 ± 0.0
250T	HGA-NoFeedback	2700	2181.6 ± 169.1	0.0 ± 0.0	0.0 ± 0.0
250T	HGA-StaticBudget	2700	2310.8 ± 199.4	808.2 ± 0.9	0.0 ± 0.0

The ablation results show that adaptive reallocation helps the scheduler satisfy partition time budgets reliably. The benefit is reflected not only in the final makespan but also in how quickly and tightly the partition time budgets are adjusted during the search.

5.7. Automotive-Characteristic DAG Case Study

To complement the scalable synthetic DAG experiments, an application-oriented case study based on automotive benchmark characteristics is included [52]. The reference benchmark does not provide a ready-to-run input model for the proposed scheduler. Instead, it reports characteristics of automotive software, including runnable activation patterns, label-based communication, execution-time distributions, and cause-and-effect chain timing requirements. These characteristics were used to construct a scheduler-compatible automotive-characteristic workload instance for the present evaluation.

The constructed workload contains 100 jobs, 212 message dependencies, and 10 end-to-end chains. Its purpose is not to reproduce a proprietary vehicle application, but to introduce an automotive-oriented dependency and timing structure into the evaluation.

The automotive-characteristic workload is evaluated using the same proposed HGA pipeline as the previous experiments, without benchmark-specific modifications. The system-level search determines allocation, execution order, deterministic route choices, and time-budget assignment, while the partition-level schedulers synthesize local task schedules under the assigned budgets. The baseline deadline for this case study is set to $D = 2600$ TU.

Table 13. Scheduling result for the automotive-characteristic workload under the baseline deadline.

Metric	Value
Deadline D	2600 TU
Final global makespan	2457 TU
Timing margin	143 TU
Positive global lateness	0 TU
Cumulative partition violation	0 TU
Deadline feasible	Yes
Partition-budget feasible	Yes
Fully feasible	Yes
First fully feasible generation	18

As shown in Table 13, the proposed HGA obtains a final global makespan of 2457 TU under the baseline deadline of 2600 TU, leaving a timing margin of 143 TU. Both the positive global lateness and the cumulative partition time-budget violation are zero. The final schedule meets the global deadline and all partition-level time-budget constraints. The first schedule that satisfies both conditions is found at generation 18, indicating that a feasible solution is obtained during the search and retained until convergence.

6. Discussion

The results show that the proposed HGA with adaptive time-budget reallocation improves the alignment between assigned partition budgets and the partition makespans realized during scheduling. In the ablation study, the proposed HGA reaches partition-budget feasibility more consistently and in fewer generations than the HGA-NoFeedback and HGA-StaticBudget variants.

An important insight is that partition-level feasibility and end-to-end feasibility must be treated as related but distinct objectives. The experiments show that adaptive time-budget negotiation is effective in repairing local infeasibility and aligning partition budgets with realized execution demand. However, once the global deadline becomes too restrictive, improving local feasibility alone is not always sufficient to remove the remaining end-to-end timing gap.

The observed budget adaptation behavior also indicates that the proposed mutation mechanism contributes to search stability, not only to feasibility repair. As the partition budgets move closer to the realized partition makespans, the hierarchical search becomes better aligned with the actual execution characteristics of the workload. This helps explain the consistent convergence behavior observed across different workload sizes and deadline settings.

The results show that the proposed HGA remains effective across both feasible and constrained scenarios. When end-to-end feasible schedules exist, the method can recover them. When they do not, it still improves the solution by removing partition-level violations and reducing the remaining global timing gap.

6.1. Limitations and Infrastructure Assumptions

Although the proposed HGA successfully computes feasible end-to-end schedules offline, its runtime temporal guarantees are entirely dependent on the enforcement capabilities of the underlying deterministic network. Establishing the practical limits of this work requires distinguishing between infrastructure assumptions that reflect current industry capabilities and those that remain future architectural requirements.

Currently, IEEE Time-Sensitive Networking (TSN) is an established, deployable standard for providing bounded latency and synchronization at the local edge and fog layers [15]. However, extending determinism across wide-area networks via IETF DetNet [16],

maintaining global synchronization via 3GPP TSaaS (Release 18) [17], and utilizing managed real-time cloud slices are currently realistic only within private, operator-managed networks. In large-scale, public deployments today, these remain future architectural requirements.

This reliance explicitly defines the boundaries of the scheduler's capabilities. The HGA treats deterministic communication as an infrastructure-provided service exposed via the Deterministic Communication Interface (DCI). Consequently, if DCI assumptions are violated—such as through unbounded routing failures, or if synchronization errors and clock drift exceed the tolerated hardware bounds—the strict timing constraints synthesized by the scheduler can no longer be mathematically guaranteed, and the affected flows degrade to best-effort execution.

To mitigate sensitivity to such timing uncertainties, the TTEFC architecture is designed with a fault-tolerance extension point (Section 3.4.3). Rather than relying on unpredictable runtime reactive scheduling, severe violations of synchronization or communication bounds trigger deterministic schedule switching via pre-computed Multi-Schedule Graphs (MSGs) [2]. Ultimately, the proposed framework successfully solves the offline scheduling problem, but explicitly separates the synthesis of these temporal budget constraints from the physical infrastructure hardware required to enforce them.

7. Conclusion

This paper presented a hierarchical deterministic scheduling framework for time-triggered edge–fog–cloud (TTEFC) architectures, addressing the fundamental challenge of providing end-to-end timing guarantees in distributed systems. The proposed approach integrates computation and communication decisions within a unified optimization model, combining partition allocation, execution ordering, deterministic routing, and time-budget assignment into a single co-design problem.

To solve this NP-hard problem efficiently, a two-level hierarchical genetic algorithm (HGA) was introduced. At the system level, global scheduling decisions are optimized, while parallel partition-level schedulers construct locally feasible schedules under assigned time budgets. A key contribution of this work is the adaptive slack reallocation operator, which enables dynamic redistribution of temporal slack across partitions, thereby improving convergence toward feasible solutions without relaxing the global deadline constraint.

The experimental results demonstrate that the proposed method consistently achieves full end-to-end schedulability under baseline deadlines, while significantly reducing global makespan compared to established baselines. Specifically, improvements of up to 46.5% over HEFT and 18.9% over Round-Robin were observed. Furthermore, the results show that the adaptive budget mechanism effectively aligns partition-level execution demand with system-level time budgets, reducing budget–makespan mismatch by up to 97.8% and stabilizing convergence behavior across different workload sizes.

The sensitivity analysis further revealed that the proposed framework remains robust under moderate deadline tightening, successfully maintaining feasibility in several constrained scenarios. Even when full end-to-end feasibility cannot be achieved due to overly restrictive deadlines, the scheduler converges to stable solutions with minimized violations and reduced global lateness. This behavior highlights the ability of the hierarchical approach to provide meaningful and predictable solutions under constrained conditions.

The results confirm that jointly optimizing computation placement, communication routing, and temporal budget constraints is essential for achieving deterministic end-to-end performance in multi-tier architectures. The proposed HGA provides a scalable and

effective solution for this co-design problem, bridging the gap between best-effort cloud scheduling and the strict requirements of safety-critical systems.

Future work will extend the proposed framework to support runtime adaptation and fault-aware scheduling by incorporating incremental metascheduling and multi-schedule graph (MSG) techniques. Additional directions include energy-aware optimization, support for heterogeneous accelerators, and validation on real TSN/DetNet platforms to strengthen the practical relevance of the approach for cyber-physical and software-defined vehicle systems.

Author Contributions: Conceptualization, O.H., J.P., and R.O.; methodology, O.H.; software, O.H.; validation, O.H., J.P., and R.O.; formal analysis, O.H.; investigation, O.H.; resources, O.H. and J.P.; data curation, O.H. and J.P.; writing—original draft preparation, O.H.; writing—review and editing, O.H., D.O., and R.O.; visualization, J.P.; supervision, D.O. and R.O.; project administration, R.O.; funding acquisition, R.O. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the EdgeAI-Trust project, which is funded within the Chips Joint Undertaking (Chips JU), a public-private partnership in collaboration with the European Union's Horizon Europe research and innovation program and National Authorities under grant agreement number 101139892.

Gefördert durch:

Data Availability Statement: The data presented in this study are available on request from the corresponding author due to privacy reasons.

Conflicts of Interest: The authors declare no conflict of interest. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript, or in the decision to publish the results.

References

1. On Edge-Fog-Cloud Collaboration and Reaping Its Benefits: A Heterogeneous Multi-Tier Edge Computing Architecture. Available online: https://www.mdpi.com/1999-5903/17/1/22 (accessed on 6 November 2025).
2. Hekal, O.; Onwuchekwa, D.; Obermaisser, R. Incremental Multi-Schedule Graph for Memory Optimization in Adaptive Time-Triggered Systems. In *Proceedings of the 2025 14th Mediterranean Conference on Embedded Computing (MECO)*, Budva, Montenegro, 2025; pp. 1–8. https://doi.org/10.1109/MECO66322.2025.11049272.
3. Alsadie, D. Advancements in heuristic task scheduling for IoT applications in fog-cloud computing: Challenges and prospects. *PeerJ Computer Science* **2024**, *10*, e2128. https://doi.org/10.7717/peerj-cs.2128.
4. Zhang, P.; Chen, N.; Xu, G.; Kumar, N.; Barnawi, A.; Guizani, M.; Duan, Y.; Yu, K. Multi-Target-Aware Dynamic Resource Scheduling for Cloud-Fog-Edge Multi-Tier Computing Network. *IEEE Transactions on Intelligent Transportation Systems* **2023**, 1–13. https://doi.org/10.1109/TITS.2023.3330419.
5. Dechouniotis, D.; Athanasopoulos, N.; Leivadeas, A.; Mitton, N.; Jungers, R.; Papavassiliou, S. Edge Computing Resource Allocation for Dynamic Networks: The DRUID-NET Vision and Perspective. *Sensors* **2020**, *20*, 2191. https://doi.org/10.3390/s20082191.
6. Oukissou, Y.; Elhaou, H.; Ait Omar, D.; Zougagh, H.; Elouaham, S. Optimized Task Scheduling in Fog-Cloud Environments Using a Cost-Aware Genetic Algorithm. **2025**.
7. Lala, J.H.; Harper, R.E. Architectural principles for safety-critical real-time applications. *Proceedings of the IEEE* **1994**, *82*, 25–40.

8. Kopetz, H.; Bauer, G. The time-triggered architecture. *Proceedings of the IEEE* **2003**, *91*, 112–126. https://doi.org/10.1109/JPROC.2002.805821.
9. Wang, S. Edge Computing: Applications, State-of-the-Art and Challenges. *Advances in Networks* **2019**, *7*, 8. https://doi.org/10.11648/j.net.20190701.12.
10. Gala, G.; Castillo Rivera, J.; Fohler, G. Work-in-Progress: Cloud Computing for Time-Triggered Safety-Critical Systems. In *Proceedings of the 2021 IEEE Real-Time Systems Symposium (RTSS)*, Dortmund, Germany, 2021; pp. 516–519. https://doi.org/10.1109/RTSS52674.2021.00054.
11. Dizdarević, J.; Carpio, F.; Jukan, A.; Masip-Bruin, X. A Survey of Communication Protocols for Internet of Things and Related Challenges of Fog and Cloud Computing Integration. *ACM Computing Surveys* **2019**, *51*, Article 116. https://doi.org/10.1145/3292674.
12. Obermaisser, R. Event-Triggered and Time-Triggered Control Paradigms. **2005**. https://doi.org/10.1007/978-0-387-23044-3.
13. Paulachan, J.; Hekal, O.; Onwuchekwa, D.; Obermaisser, R. A Time-Triggered Edge–Fog–Cloud Architecture With Hierarchical Genetic Optimization for Safety-Critical Applications. *IEEE Access* **2025**, *13*, 217406–217422. https://doi.org/10.1109/ACCESS.2025.3644152.
14. Ahmed, Z.E.; Hashim, A.A.; Saeed, R.A.; Saeed, M.M. Deterministic Networking (DetNet): Architecture, Protocols, and Advanced Applications. In *Proceedings of the 2024 9th International Conference on Mechatronics Engineering (ICOM)*, Kuala Lumpur, Malaysia, 2024; pp. 213–218. https://doi.org/10.1109/ICOM61675.2024.10652378.
15. Zhang, T.; Wang, G.; Xue, C.; Wang, J.; Nixon, M.; Han, S. Time-Sensitive Networking (TSN) for Industrial Automation: Current Advances and Future Directions. *ACM Computing Surveys* **2025**, *57*, Article 30. https://doi.org/10.1145/3695248.
16. Finn, N.; Thubert, P.; Varga, B.; Farkas, J. *Deterministic Networking Architecture*; RFC 8655; Internet Engineering Task Force: 2019. https://doi.org/10.17487/RFC8655.
17. 3rd Generation Partnership Project. *System Architecture for the 5G System (5GS); Stage 2 (Release 18)*; Technical Specification TS 23.501, Version 18.0.0; 2024. Available online: https://www.3gpp.org/specifications-technologies/releases/release-18.
18. Avan, A.; Azim, A.; Mahmoud, Q.H. A State-of-the-Art Review of Task Scheduling for Edge Computing: A Delay-Sensitive Application Perspective. *Electronics* **2023**, *12*, 2599. https://doi.org/10.3390/electronics12122599.
19. Topcuoglu, H.; Hariri, S.; Wu, M.-Y. Performance-effective and low-complexity task scheduling for heterogeneous computing. *IEEE Transactions on Parallel and Distributed Systems* **2002**, *13*, 260–274. https://doi.org/10.1109/71.993206.
20. Cao, H.; Wachowicz, M. An Edge-Fog-Cloud Architecture of Streaming Analytics for Internet of Things Applications. *Sensors* **2019**, *19*, 3594. https://doi.org/10.3390/s19163594.
21. Prokhorenko, V.; Babar, M.A. Architectural Resilience in Cloud, Fog and Edge Systems: A Survey. *IEEE Access* **2020**, *8*, 28078–28095. https://doi.org/10.1109/ACCESS.2020.2971007.
22. Alli, A.A.; Alam, M.M. The Fog Cloud of Things: A Survey on Concepts, Architecture, Standards, Tools, and Applications. *Internet of Things* **2020**, *9*, 100177. https://doi.org/10.1016/j.iot.2020.100177.
23. OpenFog Consortium Architecture Working Group. *OpenFog Reference Architecture for Fog Computing*; Technical Report OPFRA001.020817; OpenFog Consortium, 2017; pp. 1–162. Available online: https://www.iiconsortium.org/pdf/OpenFog_Reference_Architecture_2_09_17.pdf.
24. ETSI. *ETSI GS MEC 002: Mobile Edge Computing (MEC); Technical Requirements*; ETSI White Paper 1; 2016; pp. 1–40. Available online: http://www.etsi.org/standards-search.
25. Babu, E.S.; Rao, M.S.; Swain, G.; Nikhath, A.K.; Kaluri, R. Fog-Sec: Secure end-to-end communication in fog-enabled IoT network using permissioned blockchain system. *International Journal of Network Management* **2023**, *33*, e2248. https://doi.org/10.1002/nem.2248.
26. Limbasiya, T.; Das, D.; Sahay, S.K. Secure communication protocol for smart transportation based on vehicular cloud. In *Adjunct Proceedings of the 2019 ACM International Joint Conference on Pervasive and Ubiquitous Computing and Proceedings of the 2019 ACM International Symposium on Wearable Computers (UbiComp/ISWC '19 Adjunct)*, New York, NY, USA, 2019; pp. 372–376. https://doi.org/10.1145/3341162.3349305.
27. Zhang, J.; Zhu, F.; Yang, Z.; Chen, C.; Tian, X.; Guan, X. Routing and scheduling co-design for holistic software-defined deterministic network. *Fundamental Research* **2024**, *4*, 25–34.
28. Nie, H.; Wu, Y.; Zhu, W.; Zhong, J.; Yang, H.; Zhou, Y. Time-Triggered Task Offloading Scheduling in TSN-Based Edge Computing Power Networks. *IEEE Access* **2025**, *13*, 85979–85996.
29. Kakkottakath Valappil Thekkepuryil, J.; Suseelan, D.P.; Keerikkattil, P.M. An effective meta-heuristic based multi-objective hybrid optimization method for workflow scheduling in cloud computing environment. *Cluster Computing* **2021**, *24*, 2367–2384. https://doi.org/10.1007/s10586-021-03269-5.
30. Aktan, M.N.; Bulut, H. Metaheuristic task scheduling algorithms for cloud computing environments. *Concurrency and Computation: Practice and Experience* **2022**, *34*, e6513. https://doi.org/10.1002/cpe.6513.
31. Mohammed, A.S.; Mallikarjunaradhya, V.; Sreeramulu, M.D.; Boddapati, N.; Jiwani, N.; Natarajan, Y. Optimizing Real-Time Task Scheduling in Cloud-Based AI Systems Using Genetic Algorithms. In *Proceedings of the 2024 7th In-*

ternational Conference on Contemporary Computing and Informatics (IC3I), Greater Noida, India, 2024; pp. 1649–1653. https://doi.org/10.1109/IC3I61595.2024.10829055.
32. Rajashekar, K.J.; Channakrishnaraju; Gowda, P.C.; Jayachandra, A.B. SCEHO-IPSO: A Nature-Inspired Meta Heuristic Optimization for Task-Scheduling Policy in Cloud Computing. *Applied Sciences* **2023**, *13*, 10850. https://doi.org/10.3390/app131910850.
33. Al-Arasi, R.; Saif, A. Task scheduling in cloud computing based on metaheuristic techniques: A review paper. *EAI Endorsed Transactions on Cloud Systems* **2018**, *6*, 162829. https://doi.org/10.4108/eai.13-7-2018.162829.
34. Mangalampalli, S.; Karri, G.R.; Selvaraj, P. AI Enabled Resources Scheduling in Cloud Paradigm. In *6G Enabled Fog Computing in IoT*; Kumar, M.; Gill, S.S.; Samriya, J.K.; Uhlig, S., Eds.; Springer: Cham, Switzerland, 2023. https://doi.org/10.1007/978-3-031-30101-8_1.
35. Tuli, S.; Gill, S.S.; Xu, M.; Garraghan, P.; Bahsoon, R.; Dustdar, S.; Sakellariou, R.; Rana, O.; Buyya, R.; Casale, G.; Jennings, N.R. HUNTER: AI based holistic resource management for sustainable cloud computing. *Journal of Systems and Software* **2022**, *184*, 111124. https://doi.org/10.1016/j.jss.2021.111124.
36. Iftikhar, S.; Ahmad, M.M.M.; Tuli, S.; Chowdhury, D.; Xu, M.; Gill, S.S.; Uhlig, S. HunterPlus: AI based energy-efficient task scheduling for cloud–fog computing environments. *Internet of Things* **2023**, *21*, 100667. https://doi.org/10.1016/j.iot.2022.100667.
37. Chaudhary, H.; Sharma, G.; Nishad, D.K.; et al. Advanced queueing and scheduling techniques in cloud computing using AI-based model order reduction. *Discover Computing* **2025**, *28*, 75. https://doi.org/10.1007/s10791-025-09581-7.
38. Samimi, Nasim, et al. "Enabling Containerisation of Distributed Applications with Real-Time Constraints." LEIBNIZ INTERNATIONAL PROCEEDINGS IN INFORMATICS 335 (2025).
39. Samimi, N.; Nasri, M.; Basten, T.; Geilen, M. Online Admission Test for Real-Time Tasks with Arrival Curves for Server Platforms. In *Proceedings of the 32nd International Conference on Real-Time Networks and Systems (RTNS '24)*, New York, NY, USA, 2025; pp. 266–277. https://doi.org/10.1145/3696355.3696359.
40. Struhár, V.; Craciunas, S.S.; Ashjaei, M.; Behnam, M.; Papadopoulos, A.V. Hierarchical Resource Orchestration Framework for Real-Time Containers. *ACM Transactions on Embedded Computing Systems* **2024**, *23*, Article 4. https://doi.org/10.1145/3592856.
41. Friebe, A.; Cucinotta, T.; Marković, F.; Papadopoulos, A.V.; Nolte, T. Nip it in the Bud: Job Acceptance Multi-Server. In *Proceedings of the 2025 IEEE 31st Real-Time and Embedded Technology and Applications Symposium (RTAS)*, Irvine, CA, USA, 2025; pp. 26–39. https://doi.org/10.1109/RTAS65571.2025.00009.
42. Casini, D.; Abeni, L.; Marinoni, M.; Biondi, A. Runtime Monitoring for Edge Applications. In *Proceedings of the 2025 21st International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT)*, Lucca, Italy, 2025; pp. 405–412. https://doi.org/10.1109/DCOSS-IoT65416.2025.00071.
43. Nazari Cheraghlou, M.; Khadem-Zadeh, A.; Haghparast, M. A survey of fault tolerance architecture in cloud computing. *Journal of Network and Computer Applications* **2016**, *61*, 81–92. https://doi.org/10.1016/j.jnca.2015.10.004.
44. Rehman, A.U.; Aguiar, R.L.; Barraca, J.P. Fault-Tolerance in the Scope of Cloud Computing. *IEEE Access* **2022**, *10*, 63422–63441. https://doi.org/10.1109/ACCESS.2022.3182211.
45. Rao, A.P. Adaptive Fault Tolerant Resource Allocation Scheme for Cloud Computing Environment. *Journal of Organizational and End User Computing* **2021**, *33*, 136–152. https://doi.org/10.4018/JOEUC.20210901.oa7.
46. Alarifi, A.; Abdelsamie, F.; Amoon, M. A fault-tolerant aware scheduling method for fog-cloud environments. *PLoS ONE* **2019**, *14*, e0223902. https://doi.org/10.1371/journal.pone.0223902.
47. Gavrilut, V.M. *Design Optimization of IEEE Time-Sensitive Networks (TSN) for Safety-Critical and Real-Time Applications*; 2018.
48. DeRainville, F.M.; Fortin, F.A.; Gardner, M.A.; Parizeau, M.; Gagné, C. DEAP: A Python Framework for Evolutionary Algorithms. In *Proceedings of the 14th Annual Conference Companion on Genetic and Evolutionary Computation*, Philadelphia, PA, USA, 7–11 July 2012; pp. 85–92.
49. Dick, R.P.; Rhodes, D.L.; Wolf, W. TGFF: Task Graphs for Free. In *Proceedings of the Sixth International Workshop on Hardware/Software Codesign (CODES/CASHE'98)*, Seattle, WA, USA, 18 March 1998; pp. 97–101. https://doi.org/10.1109/HSC.1998.666245.
50. Stuijk, S.; Geilen, M.; Basten, T. SDF3: SDF For Free. In *Proceedings of the Sixth International Conference on Application of Concurrency to System Design (ACSD'06)*, Turku, Finland, 28–30 June 2006; pp. 276–278. https://doi.org/10.1109/ACSD.2006.23.
51. Hu, S.; Cai, Y.; Wang, S.; Han, X. Enhanced FRER Mechanism in Time-Sensitive Networking for Reliable Edge Computing. *Sensors* **2024**, *24*, 1738. https://doi.org/10.3390/s24061738.
52. Kramer, S.; Ziegenbein, D.; Hamann, A. Real World Automotive Benchmarks for Free. In *Proceedings of the 6th International Workshop on Analysis Tools and Methodologies for Embedded and Real-Time Systems (WATERS)*, 2015.